CLASSES E OBJETOS

1. Introdução à Orientação a Objetos

A Programação Orientada a Objetos (POO) busca aproximar o mundo real do mundo da programação. Em vez de focarmos apenas em funções isoladas, trabalhamos com "coisas" que possuem características e comportamentos.

2. Classe e Objeto

Classe: É o "projeto", o "molde" ou a "planta baixa". Ela define quais atributos e comportamentos algo terá. Ex: A classe Carro.

Objeto: É a instância física da classe. É o carro que está na sua garagem. É a memória alocada para aquele molde.

3. Anatomia de uma Classe

Variáveis de Instância (Atributos): São as características do objeto. Chamamos de "instância" porque cada objeto criado terá sua própria cópia dessas variáveis (um carro pode ser azul, o outro vermelho).

Métodos de Instância: São os comportamentos ou ações que o objeto pode realizar (acelerar, ligar, frear). Eles operam sobre as variáveis de instância.

4. Métodos Construtores

O construtor é o método executado no exato momento em que o objeto é criado (usando a palavra new).

Sem parâmetro: Inicia o objeto com valores padrão (ex: volume 0).

Com parâmetro: Permite que o objeto já nasça com dados específicos fornecidos pelo usuário.

5. A Palavra-chave this

O this é uma referência ao "próprio objeto". Ele resolve ambiguidades quando o nome do parâmetro do método é igual ao nome do atributo da classe (ex: this.marca = marca).

6. Tipos de Dados

Primitivos: Tipos básicos do Java que armazenam valores simples (ex: int, float, boolean, double).

Tipo Classe (Referência): Quando o tipo da variável é uma classe ou interface (ex: String, Carro, Scanner).

7. O Método toString() e a Classe Testa

toString(): Um método especial que devolve uma representação textual do objeto. Sem ele, ao imprimir um objeto, veríamos apenas o endereço de memória.

Classe Testa: É a classe que contém o método main. Ela não serve para modelar um objeto, mas sim para criar instâncias das outras classes e executar o sistema.

ENCAPSULAMENTO

1. O Conceito de Encapsulamento

Encapsular significa "colocar em uma cápsula". Na POO, é a técnica de esconder os detalhes internos de uma classe e proteger os dados contra alterações indevidas. O objetivo é a segurança e a integridade dos dados.

2. Visibilidade e a Palavra-chave private

Diferente da visibilidade public (onde qualquer um mexe em tudo), o modificador private restringe o acesso. Um atributo privado só pode ser lido ou alterado por métodos da própria classe.

3. Métodos Getters e Setters

Como os atributos são privados, usamos métodos públicos para acessá-los:

- Getters (Obter): Retornam o valor do atributo.
- Setters (Definir): Alteram o valor do atributo, mas com a vantagem de poder validar o dado (ex: não aceitar volume negativo ou marca vazia).

4. Encapsulamento de Variáveis vs. Métodos

- Variáveis: Escondemos para evitar que o saldo ou a temperatura recebam valores impossíveis.
- Métodos: Escondemos métodos que realizam lógicas internas complexas que o usuário não precisa conhecer (ex: o processo de autenticação de Wi-Fi em uma TV).

5. Setter no Construtor

Uma boa prática para fortalecer o encapsulamento é chamar o método set dentro do construtor. Assim, as regras de validação (como a marca começar com letra maiúscula) são aplicadas desde o primeiro segundo de vida do objeto.

6. Métodos de Classe (Static)

Diferente dos métodos de instância (que pertencem ao objeto), os métodos de classe são declarados com a palavra static. Eles pertencem à classe e não dependem de um objeto específico para serem chamados (ex: Math.random()).

TODO-PARTE

1. Agregação e Composição

Na Programação Orientada a Objetos, raramente uma classe trabalha sozinha. O conceito de **Associação** Todo-Parte descreve situações onde um objeto (o "Todo" ou "Container") é formado por outros objetos (as "Partes" ou "Componentes").
Essa relação é conhecida na UML como Acreção e se divide em dois tipos principais, dependendo da força do vínculo entre eles.

2. Associação de Agregação (Relação Fraca)

A Agregação representa um relacionamento onde a "Parte" pode existir independentemente do "Todo". É uma relação de compartilhamento.

- Conceito: O "Todo" utiliza a "Parte" para realizar suas tarefas, mas não é dono exclusivo dela.
- Ciclo de Vida: Independentes. Se o "Todo" for destruído, a "Parte" continua viva na memória.
- Exemplo Real: Um Time de Basquete (Todo) e seus Atletas (Partes). Se o time acabar, os atletas continuam existindo e podem jogar em outros times.

3. Associação de Composição (Relação Forte)

A Composição é uma forma estrita de agregação onde a "Parte" pertence exclusivamente ao "Todo". Existe uma dependência existencial.

- Conceito: A "Parte" não faz sentido ou não consegue existir fora do contexto do "Todo".
- Ciclo de Vida: Dependentes. Se o "Todo" for destruído, todas as suas "Partes" são destruídas automaticamente.
- Exemplo Real: Uma Nota Fiscal (Todo) e seus Itens da Nota (Partes). Não faz sentido um "Item de Nota" existir "solto" no

sistema sem estar vinculado a uma nota específica.

4. Diferenças Conceituais e de Implementação

Para facilitar o entendimento dos alunos, podemos resumir as diferenças em dois eixos:

1. Diferenças Conceituais

Característica	Agregação	Composição
Vínculo	Fraco / Geométrico	Forte / Físico
Posse	Compartilhada	Exclusiva
Exemplo	Empresa e Freelancer	Empresa e Departamentos
Símbolo UML	Losango Vazado	Losango Preenchido

2. Diferenças de Implementação (Java)

A maior diferença está em quem cria o objeto e como ele chega ao "Todo":

- Na Agregação: O objeto "Parte" é criado fora e passado para o "Todo" via parâmetro no construtor ou método set.

```
public void addAtleta(Atleta a) { this.lista.add(a); }
```

- Na Composição: O objeto "Parte" é instanciado (usando new) dentro de um método do "Todo". Os dados brutos (Strings, ints) são passados e o "Todo" fabrica a parte.

```
public void addInsumo(String nome, float preco) { this.lista.add(new Insumo(nome, preco)); }
```

4. Classe de Associação

Às vezes, um relacionamento entre dois objetos gera informações que não pertencem logicamente a nenhum dos dois isoladamente, mas sim ao vínculo entre eles.
Definição: É uma classe usada para modelar atributos e comportamentos de uma associação.
Exemplo Clássico: Passageiro e Voo.

Onde guardamos o número do "Assento"?
Se guardarmos no Passageiro, ele só poderia ter um assento na vida.
Se guardarmos no Voo, o voo só teria um passageiro.
Solução: Criamos a classe Reserva, que liga Passageiro + Voo e guarda o assento e a dataDaViagem.

4. Dicas de Ouro para o Código (This e LocalDatetime)

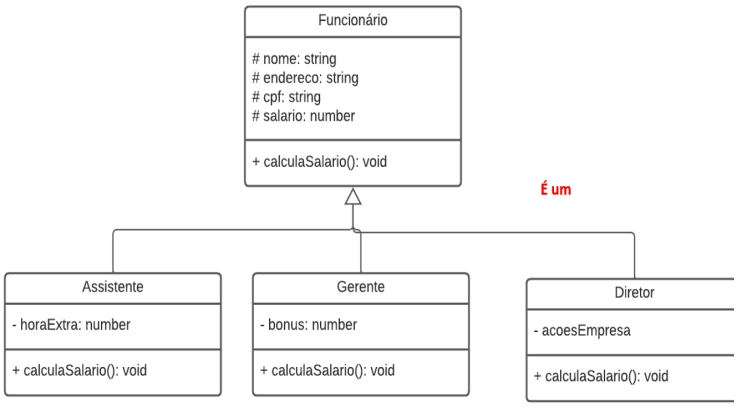
Uso do this: Sempre utilize this.atributo para deixar claro que você está acessando a variável de instância da classe, especialmente ao receber parâmetros com nomes idênticos.

Datas Modernas: Evite a classe Date. Utilize LocalDateTime.now() para capturar o momento atual e LocalDateTime.of(ano, mes, dia, hora, min) para datas específicas. Elas são imutáveis e muito mais seguras para cálculos de prazos.

HERANÇA

1. Conceito

- Associação entre classes (Generalização e Especialização)
- Classe mais genérica, chamada de classe pai ou superclasse
- Classe mais específica, chamada de classe filha ou subclasse
- Recurso das linguagens de programação orientada a objetos
- Recurso onde as classes filhas ou subclasses herdam variáveis e métodos da(s) classes(s) pai, ou superclasses(s)
- Isso promove a reutilização de código e facilita a organização hierárquica das Classes



Classe Base (ou Superclasse): Também conhecida como classe pai ou classe mãe, é a classe da qual outra classe herda. Ela contém os atributos e métodos comuns que são compartilhados pelas subclasses.
Classe Derivada (ou Subclasse): Também chamada de classe filha, é a classe que herda de outra classe. Ela pode adicionar novos atributos e métodos ou modificar os existentes, além de herdar os atributos e métodos da classe base.

Relação "é-um": A herança estabelece uma relação "é-um" entre a classe base e suas subclasses. Por exemplo, se temos uma classe "Animal" como classe base e uma classe "Cachorro" como uma de suas subclasses, podemos dizer que um cachorro é um tipo de animal.

Método de Herança: É o processo pelo qual uma classe deriva características e comportamentos de uma classe base. Isso é feito através da declaração da relação de herança na definição da classe.

Membros Privados, Protegidos e Públicos: Na herança, membros privados da classe base não são diretamente acessíveis pelas subclasses. Membros protegidos são acessíveis pelas subclasses, enquanto membros públicos são acessíveis tanto pelas subclasses quanto por classes externas.

Sobrescrita de Métodos (ou Overriding): As subclasses podem fornecer implementações específicas para métodos que foram definidos na classe base. Isso permite que as subclasses personalizem o comportamento dos métodos herdados.

Chamada ao Método da Classe Base: As subclasses podem chamar métodos da classe base usando a sintaxe apropriada, permitindo a reutilização de funcionalidades existentes.

Herança Múltipla (em algumas linguagens): Algumas linguagens de programação orientada a objetos suportam herança múltipla, permitindo que uma classe tenha mais de uma classe base. Isso pode levar a complexidades de design e ambiguidades e nem todas as linguagens oferecem suporte a essa característica.

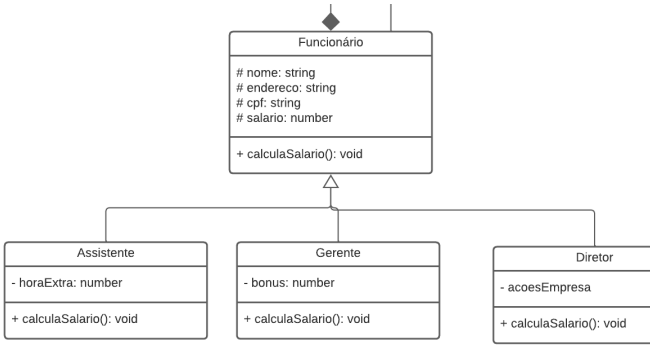
Visibilidade protegida (#)

Variável protegida é: pública para as classes filhas, privada

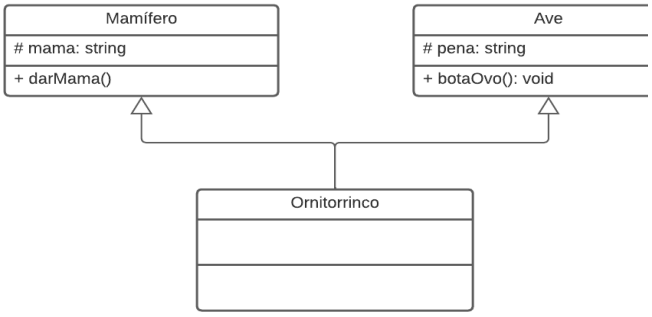
para as classes que não pertencem à hierarquia de herança

2. 2 tipos de herança

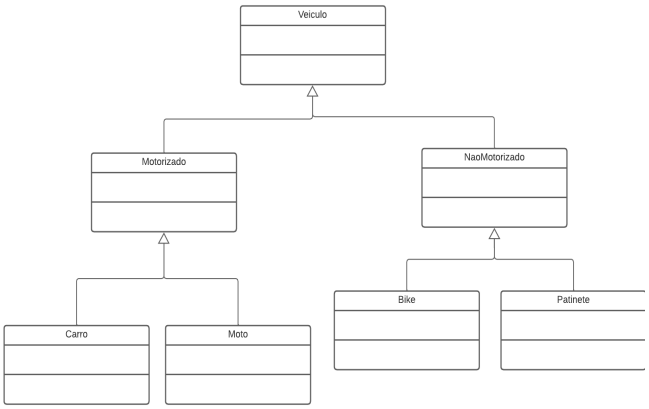
Herança simples: Classe filha ou subclasse tem apenas um único pai ou superclasse



Herança múltipla: Classe filha ou subclasse tem dois ou mais pais ou superclasses



Herança com vários níveis hierárquicos



3. Anulação de métodos

herdamos toString(), mas ele não é suficiente

vamos anular o método toString() herdado

```
toString(): string {
        return `${super.toString()} Poisonous: ${this.poisonous}`
    }
```

herdamos move(), mas não o queremos vamos anular o método move()

```
move(): string {
        return `snake crawling`
    }
```

4. Polimorfismo de herança

Em programação, o polimorfismo é a capacidade de um objeto ser referenciado de várias formas, ou seja, permitir que variáveis de um tipo genérico (classe pai) chamem métodos que se comportam de maneira específica dependendo do tipo real do objeto (classe filha).

Polimorfismo de Sobrescrita (Anulação de Método)

O que acontece: A subclasse redefine um método que já existe na classe pai usando a anotação @Override.
Quando ocorre: Em tempo de execução (Runtime). O Java descobre qual método chamar olhando para o objeto real criado na memória.
Exemplo Prático: Se você tem uma lista de Desenvolvedores e chama o método .calcularBonus(), o Java executará o cálculo de 5% para o Junior e o de 15% para o Senior automaticamente.

```
function exemploPolimorfismo(cameleao: Animal): void{
    // se a função for chamada passando uma cobra, camaleao vai representar uma cobra
    // e portanto, será executado o toString() e o move() da cobra
```

```
    // se a função for chamada passando um cavalo, camaleao vai representar um cavalo
    // e portanto, será executado o toString() e o move() do cavalo

console.log(cameleao.toString()) // polimorfismo
    console.log(cameleao.move()) // polimorfismo
}
```

5. Classe Abstrata

É uma classe que não pode ser instanciada diretamente, mas pode ser usada como uma superclasse para outras classes.
Ela serve como um esqueleto para outras classes que estendem dela, fornecendo implementações parciais ou completas de métodos, enquanto ainda permitindo que subclasses forneçam suas próprias implementações.
Uma classe abstrata é declarada usando a palavra-chave abstract antes da palavra-chave class.
Uma classe abstrata pode conter métodos abstratos, que são métodos declarados sem uma implementação.
Métodos abstratos são definidos usando a palavra-chave abstract e terminam com um ponto e vírgula, sem corpo.
Subclasses devem fornecer implementações para todos os métodos abstratos da classe abstrata.
Além de métodos abstratos, uma classe abstrata pode conter métodos concretos (ou normais) com implementações.
Os métodos concretos podem ser herdados pelas subclasses ou sobrescritos, se necessário.
Subclasses devem fornecer implementações para todos os métodos abstratos herdados da classe abstrata.

CLASSE INTERFACE

Em Java, uma interface é uma coleção de métodos abstratos e constantes (variáveis estáticas final) que podem ser implementados por classes concretas.
Elas são usadas para definir um contrato ou um conjunto de comportamentos que as classes que as implementam devem seguir.
Aqui estão alguns conceitos e fundamentos importantes relacionados às interfaces em Java:

1. Abstração

As interfaces fornecem um nível de abstração, permitindo que você defina um conjunto de métodos sem especificar a implementação real.
Isso é útil quando você deseja definir apenas o comportamento esperado de uma classe sem se preocupar com os detalhes de implementação.

2. Declaração

Uma interface é declarada usando a palavra-chave interface. Por exemplo:

```
public interface MinhaInterface{
      //metodos abstratos
      void metodo1();
      void metodo2();
}
```

3. Implementação

Uma classe pode implementar uma ou mais interfaces usando a palavra-chave implements. Por exemplo:

```
public class MinhaClasse implements MinhaInterface{
//implementação dos metodos da interface
      public void metodo1(){
            //codigo
      }
}
```

4. Herança Múltipla

Java não permite herança múltipla de classes, mas uma classe pode implementar várias interfaces. Isso permite que uma classe tenha comportamentos de várias fontes.

```
public interface I1 {
void m1();
void m2();
}
public interface I2 {
void m3();
void m4();
}
public interface I3 {
void m5();
void m6();
}
```

5. Polimorfismo

As interfaces permitem o polimorfismo. Isso significa que você pode tratar objetos de diferentes classes que implementam a mesma interface de forma uniforme.

5. Padrão de Projeto - Interface Segregation Principle (ISP):

As interfaces promovem o princípio de segregação de interface, que afirma que "nenhuma classe deve ser forçada a implementar métodos que ela não usa". Em outras palavras, é melhor ter interfaces mais específicas e coesas do que interfaces amplas e generalizadas.

6. Padrão de Projeto - Dependency Inversion Principle (DIP):

Interfaces também são fundamentais para aplicar o princípio de inversão de dependência, que incentiva o uso de abstrações (interfaces) em vez de classes concretas. Isso ajuda a desacoplar componentes do sistema, facilitando a manutenção e a extensibilidade.

7. Interfaces Funcionais

Java 8 introduziu o conceito de interfaces funcionais, que são interfaces que contêm apenas um método abstrato (método funcional) e podem ser usadas com expressões lambda. A anotação @FunctionalInterface é usada para garantir que uma interface funcional tenha apenas um método abstrato. Isso é fundamental para a programação funcional em Java.

8. Resumo

As interfaces em Java desempenham um papel crucial na definição de contratos entre diferentes partes de um sistema, promovendo a modularidade, a reutilização e a manutenibilidade do código.

Elas são uma ferramenta poderosa na caixa de ferramentas do desenvolvedor Java para criar sistemas flexíveis e extensíveis.

REGRAS DO DIAGRAMA DE CLASSES

1) As classes possuem 3 compartimentos: o nome da classe, as variáveis da classe e os métodos da classe

2) as variáveis precisam ser declaradas como

visibilidade nome_variavel: tipoVariavel

3) a visibilidade das variáveis devem ser - (privada) em todas as classes, com exceção das variáveis de superclasses (classe pai), que deve ser # (protegida)

4) os métodos precisam ser declarados como

visibilidade nome_metodo(tipo parâmetro, tipo parâmetro): tipoRetorno

5) a visibilidade dos métodos devem ser + (público) em todas as classes, com exceção dos métodos privados, que devem ser - (privado)

6) na agregação, deve-se ter um diamante não preenchido no lado da classe que representa o objeto-todo

7) na composição, deve-se ter um diamante preenchido no lado da classe que representa o objeto-todo

8) na herança, deve-se ter uma seta da classe filha para a classe pai

9) na implementação de classe interface, deve-se ter uma associação tracejada da classe que implementa apontando para a classe interface

10) o nome da classe abstrata e o nome do método abstrato devem ser escritos em itálico

SIMULADO

Questão 1: Sobre os conceitos básicos de Orientação a Objetos, qual é a principal diferença entre uma Classe e um Objeto?

a) A classe é a instância física, enquanto o objeto é o molde abstrato.
b) A classe armazena valores simples, enquanto o objeto armazena valores primitivos.
c) A classe é o "molde" ou "planta baixa" que define atributos e comportamentos, enquanto o objeto é a instância física desse molde.
d) Não existe diferença, ambos são funções isoladas no código.

Questão 2: Em relação à anatomia de uma classe, como são chamadas as características e os comportamentos do objeto, respectivamente?

a) Tipos Primitivos e Tipos de Classe.
b) Construtores e Classe Testa.
c) Referências e toString().
d) Variáveis de Instância (Atributos) e Métodos de Instância.

Questão 3: Qual é a função do Método Construtor em uma classe?

a) Devolver uma representação textual do objeto ao ser impresso.
b) Ser executado no exato momento em que o objeto é criado através da palavra-chave new.
c) Executar o método main e iniciar todo o sistema.
d) Resolver ambiguidades quando o nome do parâmetro é igual ao nome do atributo.

Questão 4: Para que serve a palavra-chave this na Programação Orientada a Objetos?

a) Para fazer uma referência ao "próprio objeto", ajudando a resolver ambiguidades entre nomes de parâmetros de métodos e atributos da classe.
b) Para iniciar um objeto com valores padrão caso não sejam passados parâmetros.
c) Para definir que uma variável armazenará um tipo de dado primitivo como int ou float.
d) Para transformar uma Classe Testa em um método de instância.

Questão 5: O que acontece se tentarmos imprimir um objeto diretamente sem utilizar o método especial toString()?

a) O sistema exibe automaticamente todos os atributos e valores do objeto.
b) O construtor é acionado novamente com parâmetros vazios.
c) Veríamos apenas o endereço de memória onde o objeto foi alocado.
d) O código apresentará um erro impedindo a criação do objeto.

O Pilar do Encapsulamento

Questão 1: O que significa o conceito de Encapsulamento na Programação Orientada a Objetos?

a) Expor todos os atributos da classe como públicos para facilitar a modificação por outras partes do código.

b) Esconder os detalhes internos de uma classe e proteger os dados contra alterações indevidas, garantindo a segurança e a integridade.
c) Criar múltiplos objetos simultâneos sem a necessidade de instanciar a classe.
d) Declarar todos os métodos e variáveis de uma classe como estáticos.

Questão 2: Qual é o efeito do modificador de visibilidade private quando aplicado a um atributo?

a) O atributo pode ser modificado por qualquer classe que herde da classe principal.
b) O atributo torna-se visível globalmente em todo o projeto.
c) O atributo tem seu acesso restrito e só pode ser lido ou alterado por métodos da própria classe.
d) O atributo é automaticamente convertido em um método da classe.

Questão 3: Qual é a função principal dos métodos Setters no contexto do encapsulamento?

a) Alterar o valor de um atributo, com a vantagem de permitir a validação do dado antes da alteração (como impedir um volume negativo).
b) Retornar o valor atual do atributo de forma segura para o usuário.
c) Esconder métodos que realizam lógicas internas complexas para evitar o mau uso da classe.
d) Iniciar o objeto com valores padrão caso o construtor não seja utilizado.

Questão 4: Por que é considerado uma boa prática chamar métodos setters dentro do método construtor?

a) Para evitar que o sistema crie novos objetos utilizando a palavra-chave new.
b) Para substituir a necessidade de criar métodos getters para aquela classe.
c) Para garantir que as regras de validação aplicadas no setter sejam exigidas desde o primeiro segundo de vida do objeto.
d) Para transformar os métodos de instância em métodos da classe.

Questão 5: Em relação aos métodos de classe declarados com a palavra static, é correto afirmar que:

a) Eles dependem obrigatoriamente da criação de um objeto para que possam ser utilizados.
b) Eles são usados exclusivamente para ocultar variáveis, como senhas e saldos.
c) Eles não podem ser usados junto com os métodos Getters e Setters.
d) Eles pertencem à classe e não dependem de um objeto específico para serem chamados (exemplo: Math.random()).

Agregação, Composição e Classe de Associação

Questão 1: Em relação ao ciclo de vida dos objetos em associações "Todo-Parte", qual é a principal diferença entre Agregação e Composição?

a) Na agregação, se o objeto "Todo" for destruído, suas "Partes" são obrigatoriamente destruídas também.
b) Na agregação, a "Parte" possui um ciclo de vida independente e continua existindo caso o "Todo" seja destruído, enquanto na composição há uma dependência existencial estrita.
c) Na composição, o "Todo" e a "Parte" possuem ciclos de vida completamente independentes, sendo uma relação de posse compartilhada.
d) Não existe diferença no ciclo de vida, ambas determinam que o objeto "Parte" pertence exclusivamente ao "Todo".

Questão 2: Como a Agregação e a Composição são representadas, respectivamente, nos diagramas da UML?

a) Agregação é representada por uma seta simples e Composição por uma linha tracejada.
b) Ambas utilizam o mesmo símbolo de um losango preenchido ().
c) Agregação utiliza um losango vazado (), enquanto a Composição utiliza um losango preenchido ().
d) Agregação utiliza um losango preenchido (), enquanto a Composição utiliza um losango vazado ().

Questão 3: Qual é a principal diferença de implementação (em código Java) entre a Agregação e a Composição?

a) Na agregação, a "Parte" deve ser obrigatoriamente instanciada dentro do construtor do "Todo".
b) Na composição, a "Parte" é sempre criada fora do "Todo" e passada inteira via parâmetro.
c) Ambas recebem instâncias prontas pelo método set, sem usar a palavra-chave new internamente.
d) Na composição, a "Parte" é instanciada (usando new) diretamente dentro de um método do "Todo", enquanto na agregação a "Parte" é criada fora e passada via parâmetro.

Questão 4: Considerando os exemplos do mundo real, qual alternativa representa corretamente um relacionamento de Composição?

a) Um Time de Basquete e seus Atletas, pois os atletas não podem existir sem o time.
b) Empresa e Freelancer, caracterizando um vínculo fraco e compartilhado.
c) Uma Nota Fiscal e seus Itens da Nota, pois não faz sentido um "Item" existir solto no sistema sem estar vinculado à sua respectiva nota.
d) Passageiro e Voo, onde a destruição do voo destrói o passageiro.

Questão 5: Em qual situação é indicado o uso de uma Classe de Associação na modelagem orientada a objetos?

a) Quando precisamos transformar uma relação fraca (Agregação) em uma relação forte e exclusiva (Composição).
b) Quando um relacionamento entre dois objetos gera informações (como o número de um assento ou uma data de viagem) que pertencem logicamente ao vínculo entre eles, e não a um deles de forma isolada.
c) Exclusivamente quando precisamos utilizar métodos com LocalDateTime no lugar da classe Date.
d) Para impedir que os atributos de uma classe fiquem expostos publicamente, protegendo os métodos getters e setters.

Herança, Polimorfismo, Classe e métodos abstratos

Questão 1: Qual é o principal benefício e objetivo do uso de Herança na Programação Orientada a Objetos?

a) Esconder os dados do objeto do acesso indevido, exigindo o uso de getters e setters.
b) Permitir que classes filhas (subclasses) herdem variáveis e métodos das classes pais (superclasses), promovendo a reutilização de código e a organização hierárquica.
c) Possibilitar a criação de objetos sem a necessidade de um método construtor.
d) Fazer com que o objeto "Parte" tenha seu ciclo de vida estritamente ligado ao objeto "Todo".

Questão 2: Qual é o modificador de visibilidade correto para permitir que as variáveis de uma superclasse (classe pai) sejam acessadas por suas classes filhas, mas continuem ocultas para o resto do sistema?

a) Privado (-), pois garante o encapsulamento total em qualquer nível.

b) Público (+), pois as filhas só herdam o que é totalmente aberto.
c) Protegido (#), pois torna a variável pública para as classes filhas, mas permanece privada para as classes que não pertencem à hierarquia de herança.
d) Estático, pois transforma a variável em um atributo da interface.

Questão 3: O que caracteriza o Polimorfismo de Sobrescrita (Anulação de Método) em tempo de execução?

a) A capacidade de uma classe ter múltiplas classes base ao mesmo tempo, herdando de todas elas.
b) A redefinição de um método da classe pai pela subclasse, permitindo que uma variável de tipo genérico chame um comportamento específico dependendo do objeto real instanciado.
c) A criação de várias classes com o exato mesmo nome para confundir o comportamento padrão do Java.
d) A obrigação de utilizar a palavra-chave abstract em todos os métodos que foram modificados.

Questão 4: Sobre as Classes Abstratas, é correto afirmar que:

a) Elas são as únicas classes que podem conter métodos com a palavra-chave static.
b) Elas devem ser instanciadas no método main para dar início à execução do programa.
c) Elas não podem ser instanciadas diretamente, servindo apenas como um "esqueleto" ou superclasse para que outras classes estendam dela.
d) Elas nunca podem possuir métodos normais com implementação, apenas métodos sem corpo.

Questão 5: O que acontece quando uma subclasse herda de uma Classe Abstrata que possui métodos abstratos (métodos declarados sem corpo/implementação)?

a) A subclasse recebe um erro e não pode ser compilada, pois herdar métodos sem corpo é proibido no Java.
b) A subclasse é obrigada a fornecer implementações (o corpo do método) para todos os métodos abstratos herdados.
c) A subclasse transforma os métodos abstratos em variáveis de instância protegidas.
d) A subclasse deve usar a anotação @FunctionalInterface para resolver a falta de implementação.

Classe Interface

Questão 1: O que é uma Interface na linguagem Java?

a) Uma classe concreta que já traz a implementação completa de todos os seus métodos para ser instanciada diretamente.
b) Uma coleção de métodos abstratos e constantes (variáveis estáticas final) que define um contrato ou um conjunto de comportamentos que as classes que as implementam devem seguir.
c) Um recurso visual utilizado exclusivamente para construir a interface gráfica (telas) de um sistema.
d) Um tipo especial de variável que substitui a criação de métodos.

Questão 2: Qual palavra-chave é utilizada na declaração de uma classe para indicar que ela adotará os comportamentos definidos por uma interface?

a) extends
b) interface
c) implements.
d) abstract

Questão 3: Em relação à quantidade de interfaces que podem ser adotadas na programação Java, é correto afirmar que:

a) Uma classe só tem permissão para implementar uma única interface por vez.
b) Interfaces só podem ser implementadas por outras interfaces, nunca por classes.
c) Nenhuma classe pode adotar uma interface sem antes estender uma classe abstrata.

d) Uma classe pode implementar uma ou mais interfaces.

Questão 4: De acordo com o Princípio de Segregação de Interface (ISP), qual é a melhor prática ao criar interfaces?

a) Criar interfaces amplas e generalizadas para que uma única interface sirva para todo o sistema.

b) Criar interfaces mais específicas e coesas, pois nenhuma classe deve ser forçada a implementar métodos que ela não usa.

c) Evitar o uso de interfaces em favor de depender diretamente de classes concretas.

d) Fazer com que todas as interfaces tenham as exatas mesmas assinaturas de método de uma superclasse.

Questão 5: Introduzido no Java 8, o conceito de "Interface Funcional" (garantido pela anotação @FunctionalInterface) determina que:

a) A interface não pode ter nenhum método declarado em seu interior.

b) A interface substitui o uso das classes abstratas clássicas no momento da compilação.

c) A interface contém apenas um método abstrato (método funcional) e pode ser usada com expressões lambda.

d) A interface permite a implementação múltipla de métodos completos sem depender de classes.

Diagrama de Classes

Questão 1: De acordo com as regras de diagramas, quantos e quais são os compartimentos padrão que representam uma classe?

a) Dois compartimentos: o nome da classe e seus atributos.

b) Quatro compartimentos: nome, variáveis, métodos e pacotes.

c) Três compartimentos: o nome da classe, as variáveis da classe e os métodos da classe.

d) Três compartimentos: o nome do projeto, as variáveis públicas e os métodos estáticos.

Questão 2: Qual deve ser a visibilidade padrão para a declaração de variáveis em uma classe comum e em uma superclasse (classe pai), respectivamente?

a) Pública (+) para todas as classes, para facilitar a herança.

b) Privada (-) em todas as classes em geral, com exceção das variáveis de superclasses que devem ser protegidas (#).

c) Protegida (#) em todas as classes, exceto nas classes filhas que devem ser privadas (-).

d) Nenhuma visibilidade precisa ser declarada explicitamente nos diagramas de classes.

Questão 3: Como as relações de Agregação e Composição devem ser representadas no diagrama?

a) A agregação usa uma linha tracejada e a composição usa uma seta sólida.

b) A agregação usa um diamante preenchido no lado da "parte", enquanto a composição usa um vazado.

c) A agregação usa um diamante não preenchido e a composição usa um diamante preenchido, ambos posicionados do lado da classe que representa o objeto-todo.

d) Ambas utilizam uma seta simples apontando para a classe principal.

Questão 4: Como deve ser ilustrada a implementação de uma Classe Interface por outra classe?

a) Com uma seta sólida apontando da classe que implementa para a classe pai.

b) Através de uma associação tracejada que sai da classe que implementa e aponta para a classe interface.

c) Utilizando um losango na extremidade da linha que se conecta à interface.

d) Não existe representação gráfica para interfaces nos diagramas de classe.

Questão 5: Qual é a regra de formatação exigida para escrever o nome de uma classe abstrata e de um método abstrato no diagrama?
a) Ambos devem ser escritos em letras maiúsculas.
b) O nome da classe deve estar em negrito e o do método sublinhado.
c) Tanto o nome da classe abstrata quanto o nome do método abstrato devem ser escritos em itálico.
d) Devem vir acompanhados da palavra-chave abstract antes de seus nomes.

TODO-PARTE

AGREGAÇÃO

Métodos de Classe (Static)

A Agregação é um tipo de relacionamento "Todo-Parte" onde a parte pode existir sem o todo. É uma relação fraca de dependência.

1. O Conceito de "Tem-um"

Diferente da Herança (onde um objeto é algo), na Agregação um objeto possui uma referência a outro.
Exemplo: Uma Equipe possui vários Profissionais. Se a equipe for desfeita, os profissionais ainda existem.

2. Implementação Técnica

Na prática, a Agregação acontece quando uma classe tem um atributo que é do tipo de outra classe (Tipo Classe/Referência).
Coleções: É muito comum usar List ou ArrayList para agregar vários objetos dentro de um só.
Independência: O objeto "parte" geralmente é criado fora da classe "todo" e passado para ela via construtor ou método setter.

3. Anatomia da Agregação

Todo (Agregador): A classe principal que contém a lista ou a referência (ex: UBS).
Parte (Agregado): A classe que é contida (ex: Paciente).
Multiplicidade: Define quantos objetos estão envolvidos (1 para 1, 1 para muitos, etc.).

4. Vantagens no Desenvolvimento

Reutilização: Você pode usar a classe Endereco tanto em Paciente quanto em UnidadeSaude.
Manutenção: Se a regra de como um Medico funciona mudar, você só mexe na classe Medico, e a classe Hospital (que o agrega) continua funcionando normalmente.

CLASSE DE ASSOCIAÇÃO

Diferente de uma associação simples, a Classe de Associação é usada quando o relacionamento em si precisa armazenar informações ou executar comportamentos.

1. Quando o Relacionamento Ganha Vida

Atributos Próprios: Quando a união de dois objetos gera novos dados.
Exemplo: A relação entre Cidadão e Vacina gera a Aplicação (que tem data, lote e dose). Sem a aplicação, a data não faz sentido sozinha em nenhum dos dois.

2. Transformando Relação em Objeto

Em vez de apenas uma linha ligando as classes no diagrama (UML), penduramos uma classe nessa linha com uma linha pontilhada.
No código Java, a classe de associação vira uma classe comum que contém referências para os dois objetos que ela une.

3. Anatomia da Classe de Associação

Referência A: O objeto do primeiro lado (ex: Paciente paciente).
Referência B: O objeto do segundo lado (ex: Medico medico).
Dados do Vínculo: Atributos específicos daquela ocorrência (ex: String diagnostico, LocalDate data).

COMPOSIÇÃO

1. O que é Composição?

A composição é um princípio de design onde uma classe "tem um" ou "é composta por" objetos de outras classes. Em vez de criar uma classe gigante com centenas de atributos, nós dividimos o sistema em partes menores e mais lógicas que se relacionam.
Conceito chave: Relação de "Tem um" (A Escola tem um Endereço, o Carro tem um Motor).

2. Por que usar?

Reuso de Código: Você escreve a classe Endereco uma vez e a utiliza na classe Aluno, Professor e Empresa.
Organização: O código fica mais limpo e fácil de manter. Se a lógica de como validar um CEP mudar, você só mexe na classe Endereco.

3. Como funciona na prática

Na anatomia da classe, um dos atributos deixa de ser um tipo primitivo (como int ou String) e passa a ser o nome de outra classe que você criou.

```
public class Carro {
    String modelo;
    Motor motor; // Atributo que é um objeto de outra classe (Composição)
}
```

4. O Ciclo de Vida (Dependência)

Na composição "estrita", o objeto-filho geralmente depende do objeto-pai para fazer sentido. Se você deletar um Pedido, os ItensDoPedido que pertenciam a ele geralmente deixam de existir ou perdem o sentido no contexto do sistema.

5. Composição vs. Herança

Muitos programadores iniciantes confundem os dois, mas a regra de ouro é:
Herança (É um): Um Gato é um Animal. (Usa o extends).
Composição (Tem um): Um Gato tem um Dono. (Usa um atributo do tipo Pessoa).

6. Inicialização na Composição

Existem duas formas principais de inicializar objetos compostos:
No Construtor: Você passa o objeto pronto como parâmetro.

```
public Carro(Motor m) { this.motor = m; }
```

Instanciação Interna: A classe principal cria o objeto interno sozinha.

```
public Carro() { this.motor = new Motor(); }
```

7. Acessando Membros Compostos

Para acessar um método de um objeto que está dentro de outro, usamos a técnica de "encadeamento" de pontos.

```
meuCarro.getMotor().ligar();
```

Aqui, meuCarro acessa o atributo motor, e a partir dele chamamos o método ligar() que pertence à classe Motor.

---

1. Relacionamentos de Todo-Parte (Hierarquia de Objeto)

Esses relacionamentos descrevem situações onde uma classe é composta por outras. A regra de ouro aqui é: "A parte está contida no todo".

- Exemplo: Uma Faculdade (Todo) possui várias Salas de Aula (Parte).
- Pergunta de Teste: "Sala de aula é uma faculdade?" Não. Ela apenas pertence à faculdade.

Representação em UML

(Diamante)Agregação (diamanete aberto ◇): Indica um relacionamento mais fraco. A parte pode existir independentemente do todo.

Composição (Diamante Preenchido ◆): Indica um relacionamento forte. Se o "todo" for destruído, as "partes" também deixam de existir.

2. Generalização e Especialização (Hierarquia de Classe)

Este conceito lida com categorias e subcategorias.

- Generalização: É o conceito da Classe Pai (ou Superclasse). É a visão mais genérica.
- Especialização: É o conceito da Classe Filha (ou Subclasse). É a visão mais específica.

Exemplo: Funcionário

- Classe Pai (Genérica): Funcionário.
- Classes Filhas (Específicas): Gerente, Assistente, Diretor.
- Pergunta de Teste ("É um?"): Um Gerente é um Funcionário? Sim. Esta afirmação deve ser sempre verdadeira.

3. Herança e Reuso

A herança é o mecanismo que permite a implementação da Generalização/Especialização.

Processo: A classe filha herda automaticamente variáveis (atributos) e métodos da classe pai.

Objetivo (Reuso): Permite a reutilização de código, evitando que você precise escrever as mesmas funções em todas as classes filhas.

Direção da Seta: No diagrama, a seta sempre aponta da classe específica (filha) para a classe pai.

Nota: a classe filha consegue usar os métodos da classe pai, mas o contrário não ocorre (a classe pai não conhece as especializações da filha).

4. Modificadores de Acesso (Visibilidade)

Os modificadores controlam o nível de exposição dos dados, garantindo o princípio do Encapsulamento.

1. **Pública** `+`

Definição: A classe, seus métodos e atributos são acessíveis por qualquer outra classe do sistema. Não há restrições.

2. **Privada** `-`

Definição: Os membros são acessíveis apenas dentro da própria classe onde foram criados. Nenhuma outra classe (nem mesmo as filhas) tem acesso direto.

3. **Protegida** `#`

Definição: É o meio-termo, funcionando como uma "mistura" de pública e privada:

Comportamento de Pública: É visível para as Classes Filhas (Subclasses) e classes do mesmo pacote.

Comportamento de Privada: É invisível para todas as demais classes externas que não possuem relação de herança.

Ele permite que o filho use o que é do pai sem expor esses dados para o resto do mundo.

Modificador	Símbolo	quem acessa?
Público	+	Todo o sistema
Protegido	#	A própria classe e suas Classes Filhas.
Privado	-	Apenas a própria classe.

EXE. CLASSE E OBJETO

Exe 1: Sistema de Gerenciamento de Conta Bancária

Objetivo: Criar uma aplicação em Java para simular o funcionamento de uma conta bancária simplificada, reforçando os conceitos de classes, objetos, atributos e métodos.

Parte 1: A Classe Conta

Crie uma classe chamada Conta (no arquivo Conta.java) com as seguintes especificações:

1. **Atributos (Visibilidade Pública):**
    - numeroConta (int)
    - agencia (int)
    - nomeCliente (String)
    - saldo (float)
    - status (boolean) — *Deve ser true se a conta estiver ativa e false se estiver encerrada.*
2. **Método Construtor:**
    - Deve receber como parâmetro o número, a agência e o nome do cliente.
    - O saldo deve iniciar em 0 e o status deve iniciar como true (ativa).
3. **Métodos de Comportamento:**
    - depositar(float valor): Só deve permitir o depósito se a conta estiver ativa. Se estiver ativa, soma o valor ao saldo e exibe uma mensagem de confirmação.
    - sacar(float valor): Só deve permitir o saque se a conta estiver ativa e se o saldo atual for maior ou igual ao valor solicitado. Caso contrário, exibe uma mensagem de erro ("Saldo insuficiente" ou "Conta inativa").
    - encerrarConta(): Altera o status da conta para false apenas se o saldo estiver zerado. Se houver dinheiro na conta, deve avisar que o cliente precisa sacar o valor antes de encerrar.
    - toString(): Deve retornar uma String formatada contendo todos os dados da conta (Número, Agência, Cliente, Saldo atual e se está Ativa ou Encerrada).

Parte 2: A Classe de Teste

Crie uma classe chamada TestaConta (no arquivo TestaConta.java) com o método main para realizar as seguintes ações:

1. Instancie um objeto c1 para o cliente "João Silva".
2. Tente realizar um depósito de $R\$ 500,00$.
3. Tente realizar um saque de $R\$ 200,00$ e imprima o estado da conta.
4. Instancie um objeto c2 para a cliente "Maria Souza".
5. Tente sacar $R\$ 50,00$ (sem ter saldo) e verifique a mensagem de erro.
6. Tente encerrar a conta c1 enquanto ela ainda tem saldo e veja o que acontece.
7. Saque o restante do dinheiro de c1, encerre a conta e

imprima os dados finais para confirmar o status.

Conta.java

```
package Conta;

public class Conta{
    public int numeroConta, agencia;
    public String nomeCliente;
    public float saldo;
    public boolean status;

    public Conta(int numeroConta, int
agencia, String nomeCliente){
        this.numeroConta = numeroConta;
        this.agencia = agencia;
        this.nomeCliente = nomeCliente;
        this.saldo = 0;
        this.status = true;
    }

    public void depositar(float valor){
        if (this.status){
            this.saldo += valor;
            System.out.println("- Depósito
realizado.");
        } else{
            System.out.println("*
Conta.Conta inativa *");
        }
    }

    public void sacar(float valor){
        if (this.status){
            if (this.saldo >= valor){
                this.saldo -= valor;
                System.out.println("-
Saque realizado. Novo saldo: $" +
this.saldo);
            } else {
                System.out.println("*
Saldo indisponível *");
            }
        } else{
            System.out.println("*
Conta.Conta inativa *");
        }
    }

    public void encerrar(){
        if (this.status && this.saldo ==
0){
            this.status = false;
            System.out.println("-
Conta.Conta encerrada.");
        } else{
            System.out.println("* Para
encerrar a conta retire o saldo restante
*");
        }
    }

    public String toString(){
        return
"-------------------------" +
                "\nNúmero: " +
this.numeroConta +
                "\nAgência: " +
this.agencia +
                "\nCliente: " +
this.nomeCliente +
                "\nSaldo atual: " +
this.saldo +
                "\nStatus da conta: " +
this.status +

"\n-------------------------";
    }

}
```

TestaConta.java

```
package Conta;

public class TestaConta {
    public static void main(String[]
args){
        Conta c1 = new Conta(123, 10,
"João Silva");

        c1.depositar(500);
        c1.sacar(200);
        System.out.println(c1.toString());

        Conta c2 = new Conta(321, 11,
"Maria Souza");
        System.out.println(c2.toString());
        c2.sacar(50);

        c1.encerrar();
        c1.sacar(c1.saldo);
        c1.encerrar();
        System.out.println(c1.toString());
    }
}
```

Exercício 2: Sistema de Streaming (Modelando Assinaturas)

Contexto: Os alunos devem criar uma classe que gerencie o status de um perfil em uma plataforma de vídeo (como a Netflix).

A Classe Streaming

1. **Atributos:**
    - usuario (String)
    - plano (String: "Básico", "Premium" ou "Família")
    - mensalidade (float)
    - ativo (boolean)

- ultimoFilmeAssistido (String)

2. **Construtor:**
    - Deve receber usuario e plano.
    - O valor da mensalidade deve ser definido automaticamente no construtor com base no plano: Básico = 25.90, Premium = 45.90, Família = 60.90.
    - O status ativo inicia como true e ultimoFilmeAssistido inicia vazio.
3. **Métodos:**
    - assistirFilme(String nomeFilme): Se a conta estiver ativa, atualiza o ultimoFilmeAssistido e imprime "Assistindo: [nome]". Se estiver inativa, avisa que precisa pagar a fatura.
    - cancelarAssinatura(): Muda o status ativo para false.
    - toString(): Exibe todos os dados, formatando o status de "Ativo" para "Suspenso" se for falso.

Streaming.java

```
package Streaming;

public class Streaming{
    public String usuario;
    public boolean status;
    public String ultimoFilmeAssistido;
    public Plano plano;

    public Streaming(String usuario,
String plano){
        this.usuario = usuario;
        this.plano = new Plano(plano);
        this.status = true;
        this.ultimoFilmeAssistido = "";
    }

    public Plano getPlano(){
        return this.plano;
    }

    public String getStatus(){
        if (this.status){
            return "Ativo";
        }else{
            return "Suspenso";
        }
    }
    public void assistirFilme(String
nomeFilme){
        if (this.status){
            this.ultimoFilmeAssistido =
nomeFilme;
            System.out.println("-
Assintindo: " + nomeFilme);
        } else{
            System.out.println("* Conta
inativa *");
        }
    }
    public void cancelarAssinatura(){
        this.status = false;
    }
    public void reativarAssinatura(){
        this.status = true;
    }
    public String toString(){
        return "----------------" +
                "\nUsuario: " +
this.usuario +
                "\nStreaming.Plano: " +
this.getPlano().tipo +
                "\nMensalidade " +
this.getPlano().mensalidade +
                "\nUltimo filme assistido:
" + this.ultimoFilmeAssistido +
                "\nStatus: " +
this.getStatus() +
                "\n----------------";
    }

}
```

Plano.java

```
package Streaming;

public class Plano {
    public String tipo;
    public double mensalidade;

    public Plano(String tipo){
        this.tipo = tipo;
        if(tipo.equals("Básico")){
            this.mensalidade = 25.9;
        } else if (tipo.equals("Premium"))
{
            this.mensalidade = 45.9;
```

```
        } else if (tipo.equals("Família"))
{
            this.mensalidade = 60.9;
        }
    }

}
```

TestaSteaming.java

```
package Streaming;

public class TestaStreaming {
    public static void main(String[]
args){
        Streaming c1 = new
Streaming("José", "Família");

        System.out.println(c1.toString());
        c1.assistirFilme("La La Land");
        c1.cancelarAssinatura();
        System.out.println(c1.toString());
    }
}
```

Exercício 3: Painel de Jogo de Basquete (Controle de Pontuação)

Contexto: Como o basquete é um esporte muito forte na região, vamos modelar um placar eletrônico para uma partida.

A Classe Placar

1. **Atributos:**
    - nomeTimeCasa (String)
    - nomeTimeVisitante (String)
    - pontosCasa (int)
    - pontosVisitante (int)
    - periodoQuarto (int) — *Representa os 4 quartos do jogo.*
2. **Construtor:**
    - Recebe os nomes dos dois times.
    - Inicia as pontuações em 0 e o periodoQuarto em 1.
3. **Métodos:**
    - registrarPonto(String time, int tipo): O parâmetro time diz qual time pontuou ("casa" ou "visitante"). O parâmetro tipo recebe o valor do ponto (1 para lance livre, 2 para cesta normal ou 3 para cesta de três). O método deve somar ao atributo correspondente.
    - proximoQuarto(): Incrementa o periodoQuarto. Se já estiver no 4, deve avisar que o jogo terminou.
    - toString(): Exibe o placar atual: "TIME A [pontos] x [pontos] TIME B - Período: [quarto]".

Placar.java

```
package exe3;
public class Placar {
    // Atributos
    private String nomeTimeCasa;
    private String nomeTimeVisitante;
    private int pontosCasa;
    private int pontosVisitante;
    private int periodoQuarto;
    // Construtor
    public Placar(String nomeTimeCasa,
String nomeTimeVisitante) {
        this.nomeTimeCasa = nomeTimeCasa;
        this.nomeTimeVisitante =
nomeTimeVisitante;
        this.pontosCasa = 0;
        this.pontosVisitante = 0;
        this.periodoQuarto = 1;
    }
    // Método para registrar pontos (com
validação de time e tipo)
    public void registrarPonto(String
time, int tipo) {
        // O basquete só tem pontos de 1,
2 ou 3
        if (tipo < 1 || tipo > 3) {
            System.out.println("Pontuação
inválida!");
            return;
        }
        if (time.equalsIgnoreCase("casa"))
{
            this.pontosCasa += tipo;
```

```
            System.out.println("Cesta de "
+ tipo + " pontos para o " +
nomeTimeCasa);
        } else if
(time.equalsIgnoreCase("visitante")) {
            this.pontosVisitante += tipo;
            System.out.println("Cesta de "
+ tipo + " pontos para o " +
nomeTimeVisitante);
        } else {
            System.out.println("Time não
identificado.");
        }
    }
    // Método para avançar o tempo de jogo
    public void proximoQuarto() {
        if (this.periodoQuarto < 4) {
            this.periodoQuarto++;
            System.out.println("Iniciando
o " + this.periodoQuarto + "° Quarto!");
        } else {
            System.out.println("O jogo já
terminou!");
        }
    }
    @Override
    public String toString() {
        return nomeTimeCasa + " [" +
pontosCasa + "] x [" + pontosVisitante +
"] " +
                nomeTimeVisitante + " -
Período: " + periodoQuarto;
    }
}
```

TestaPlacar.java

```
package exe3;
public class TestaPlacar {
    public static void main(String[] args)
{
        // Instanciando o placar
        Placar jogo = new Placar("Sesi
Franca", "Bauru");
        System.out.println("--- Início de
Jogo ---");

System.out.println(jogo.toString());

        // Simulando pontuações
        jogo.registrarPonto("casa", 3);
// Cesta de 3 do Franca
        jogo.registrarPonto("visitante",
2);  // Cesta de 2 do Bauru
        jogo.registrarPonto("casa", 1);
// Lance livre do Franca

        // Avançando o jogo
        jogo.proximoQuarto();
        jogo.proximoQuarto();
        jogo.proximoQuarto(); // Chegamos
ao 4° Quarto
        jogo.registrarPonto("casa", 2);
        System.out.println("\n--- Placar
Final ---");
System.out.println(jogo.toString());
        // Tentando avançar após o fim
        jogo.proximoQuarto();
    }
}
```

EXE. ENCAPSULAMENTO

Exercício 1: Sistema de Ar-Condicionado Inteligente

Contexto: Os alunos devem modelar um Ar-Condicionado que possui limites rígidos de temperatura e um processo de resfriamento que depende de uma verificação interna.

A Classe ArCondicionado

1. **Atributos Privados:** marca (String), modelo (String), temperatura (int) e ligado (boolean).
2. **Construtores:** Um vazio (padrão) e um que receba todos os parâmetros (usando os setters).
3. **Encapsulamento (Setters com regras):**
    - setTemperatura(int t): Só deve aceitar valores entre **16 e 30** graus. Caso contrário, exibir "Temperatura fora da faixa".
    - setMarca(String m): Só aceita se a marca tiver pelo menos 3 caracteres.
4. **Método Público ativarModoTurbo():**
    - Este método deve chamar o método privado verificarCompressor().
    - Se o compressor estiver "Ok", a temperatura deve ser ajustada automaticamente para 16 (usando o setter).
5. **Método Privado verificarCompressor():**

- Deve simular uma checagem (usando Math.random()). Se o número for maior que 2, retorna true, caso contrário false (simulando falha técnica).

6. **Método toString():** Exibir os dados e se o aparelho está ligado.

ArCondicionado.java

```
public class ArCondicionado {
    // Atributos Privados
    private String marca, modelo;
    private int temperatura;
    private boolean ligado;

    // Construtor sem parâmetros
    public ArCondicionado() {
        this.marca = "Generico";
        this.modelo = "Standard";
        this.temperatura = 24;
        this.ligado = false;
    }

    // Construtor com parâmetros
    public ArCondicionado(String marca,
String modelo, int temperatura) {
        this.setMarca(marca);
        this.modelo = modelo;
        this.setTemperatura(temperatura);
// Usa o setter para validar a temp
inicial
        this.ligado = false;
    }

    // Getters e Setters
    public String getMarca() { return
marca; }
    public void setMarca(String marca) {
        if (marca.length() >= 3){
            this.marca = marca;
        }
        else System.out.println("Marca
inválida");
    }

    public String getModelo() { return
modelo; }
    public void setModelo(String modelo) {
this.modelo = modelo; }

    public int getTemperatura() { return
temperatura; }

    public void setTemperatura(int
temperatura) {
        // Regra de Ouro: Validação de
faixa de temperatura
        if (temperatura >= 16 &&
temperatura <= 30) {
            this.temperatura =
temperatura;
        } else {
            System.out.println("Erro:
Temperatura " + temperatura + "°C inválida
(Permitido: 16°C a 30°C).");
        }
    }

    public boolean isLigado() { return
ligado; }
    public void setLigado(boolean ligado)
{ this.ligado = ligado; }

    // Métodos Públicos
    public void ligar() {
        if (!this.ligado) {
            this.ligado = true;
            System.out.println("Aparelho
ligado.");
        }
    }

    public void desligar() {
        if (this.ligado) {
            this.ligado = false;
            System.out.println("Aparelho
desligado.");
        }
    }

    public void ajustarTemperatura(int
novaTemp) {
        if (this.ligado) {
            this.setTemperatura(novaTemp);
        } else {
            System.out.println("Aviso:
Ligue o aparelho antes de ajustar a
temperatura.");
        }
    }

    public void modoTurbo() {
        if (this.ligado) {

this.processarResfriamentoRapido();
        } else {
            System.out.println("Aviso: Não
é possível ativar Modo Turbo com o
aparelho desligado.");
        }
    }

    // Método Privado (Lógica interna
escondida)
    private void
processarResfriamentoRapido() {
        System.out.println("Aumentando
rotação do compressor...");
        System.out.println("Ligando
ventilação máxima...");
        this.setTemperatura(16); // Força
a temperatura mínima permitida
```

```
    }

    @Override
    public String toString() {
        return "ArCondicionado [" + marca
+ " " + modelo +
                " | Temp: " + temperatura
+ "°C" +
                " | Status: " + (ligado ?
"Ligado" : "Desligado") + "]";
    }
}
```

TestaAr.java

```
public class TestaAr {
    public static void main(String[] args)
{
        // 1. Instância com marca/modelo
preferidos e 22°C
        ArCondicionado meuAr = new
ArCondicionado("Samsung", "WindFree", 22);

        // 2. Teste de temperatura
inválida e válida
        System.out.println("--- Testando
ajustes de temperatura ---");
        meuAr.ajustarTemperatura(10); //
Deve dar erro (aparelho desligado + valor
baixo)
        meuAr.ligar();                //
Precisamos ligar para ajustar
        meuAr.ajustarTemperatura(10); //
Deve dar erro (valor fora da faixa)
        meuAr.ajustarTemperatura(25); //
Deve funcionar

        // 3. Imprimir dados
        System.out.println("\nEstado
atual: " + meuAr.toString());

        // 4. Testar modoTurbo com
aparelho desligado
        System.out.println("\n--- Testando
Modo Turbo ---");
        meuAr.desligar();
        meuAr.modoTurbo();

        // 5. Ligar, usar modoTurbo e
verificar estado final
        meuAr.ligar();
        meuAr.modoTurbo();
        System.out.println("\nEstado
Final: " + meuAr.toString());
    }
}
```

Exercício 2: Sistema de Drone Profissional

Contexto: Vamos modelar o controle de um Drone. Aqui, a segurança é fundamental, então a validação de altura é o foco do encapsulamento.

A Classe Drone

1. **Atributos Privados:** codigo (String), altura (float), bateria (int) e emVoo (boolean).
2. **Encapsulamento (Setters com regras):**
    - setAltura(float a): O drone não pode voar abaixo de 0 nem acima de **120 metros** (regulamentação). Exibir aviso se for inválido.
    - setBateria(int b): Deve estar entre 0 e 100.
3. **Método Público decolar():**
    - Só pode decolar se a bateria estiver acima de 20%.
    - Deve chamar o método privado testarMotores(). Se os motores estiverem prontos, mudar emVoo para true e definir altura para 2 metros.
4. **Método Privado testarMotores():**
    - Exibe mensagens: "Testando hélices...", "Calibrando GPS...".
    - Gera um random. Se for menor que 8, os motores estão prontos.
5. **Métodos de Voo:** subir(float x) e descer(float x) que utilizam o setAltura para somar/subtrair da altura atual.

Drone.java

```
package DroneProfissional;
```

```
public class Drone {
    private String codigo;
    private float altura;
    private int bateria;
    private boolean emVoo;

    public Drone(){}

    public Drone(String codigo, float
altura, int bateria){
        this.codigo = codigo;
        this.setAltura(altura);
        this.setBateria(bateria);
        this.emVoo = false;
    }

    public void setAltura(float altura){
        if (altura >= 0 && altura <= 120){
            this.altura = altura;
        } else System.out.println("Altura
inválida");
    }

    public void setBateria(int bateria){
        if (bateria >= 0 && bateria <=
100){
            this.bateria = bateria;
        } else System.out.println("Valor
inválido");
    }

    public void decolar(){
        if (this.bateria > 20 &&
this.testarMotores()){
            this.emVoo = true;
            this.setAltura(2);

System.out.println("Decolando...");
        } else System.out.println("Não foi
possível decolar");
    }

    private boolean testarMotores(){
        System.out.println("Testando
hélices...\nCalibrando GPS...");
        int random = (int)
(Math.random()*10);
        return random < 8;
    }

    public void subir(float x){
        this.setAltura(this.altura + x);
    }

    public void descer(float x){
        this.setAltura((this.altura - x));
    }

    public String toString(){
        return
"--------------------------" +
                "\nCódigo: " + this.codigo
+
                "\nAltura: " + this.altura
+
                "\nBateria: " +
this.bateria +
                "\nSituação: " +
((this.emVoo) ? "Voando":"No chão") +

"\n--------------------------";
    }
}
```

TestaDrone.java

```
package DroneProfissional;

public class TestaDrone {
    public static void main(String[]
args){
        Drone d1 = new Drone("1234", 0,
100);

        d1.decolar();
        System.out.println(d1.toString());

        Drone d2 = new Drone("4321", -1,
15);
        d2.decolar();

        d1.subir(50);
        System.out.println(d1.toString());

        d1.descer(60);
    }
}
```

EXE TODO-PARTE

Exercicio 0: Sistema de Gestão de Disciplinas

Objetivo: Implementar um sistema que gerencie uma Disciplina, seus Alunos e as Avaliações realizadas.

1. Classe Aluno (Objeto Independente)

Crie a classe Aluno com os atributos: id, nome, ra (Registro Acadêmico) e curso.

- Adicione construtores, getters/setters e toString().

2. Classe Questao (Objeto Parte)

Crie a classe Questao com os atributos: numero, enunciado e valor (float).

- Adicione construtores, getters/setters e toString().

3. Classe Avaliacao (Composição)

Uma Avaliacao é composta por várias questões. Se a avaliação for excluída, as questões perdem o sentido.

- Atributos: id, nome (ex: "P1", "Trabalho 1") e um ArrayList<Questao>.
- **Método de Composição:** void adicionarQuestao(int num, String texto, float peso). Note que este método recebe os dados e cria o objeto Questao internamente.

4. Classe Disciplina (Agregação e Composição)

A classe Disciplina será o nosso "Todo" principal.

- Atributos: id, nome, professor (String) e dois ArrayLists: um para Aluno e outro para Avaliacao.
- **Regras de Relacionamento:**
    1. **Agregação:** Implemente void matricularAluno(Aluno a). O aluno existe fora da disciplina e é apenas vinculado a ela.
    2. **Composição:** Implemente void criarAvaliacao(int id, String nome). A avaliação nasce dentro da disciplina.

5. Classe de Teste TestaAcademico

No método main:

1. Instancie 2 objetos Aluno (eles devem ser independentes).
2. Instancie um objeto Disciplina (ex: "Programação Orientada a Objetos").
3. **Agregação:** Matricule os alunos criados na disciplina.
4. **Composição:** Crie uma avaliação chamada "Prova Semestral" dentro da disciplina.
5. **Composição aninhada:** Dentro da avaliação criada, adicione 3 questões usando o método específico.
6. Imprima o toString() da disciplina para visualizar toda a árvore de objetos.

Aluno.java

```
package GestaoDisciplinas;

public class Aluno {
    private int id;
    private String nome, ra, curso;

    public Aluno() {
    }
    public Aluno(int id, String curso,
String nome, String ra) {
        this.id = id;
        this.curso = curso;
        this.nome = nome;
        this.ra = ra;
    }
    public int getId() {
        return id; }
    public void setId(int id) {
        this.id = id;}
    public String getCurso() {
        return curso; }
    public void setCurso(String curso) {
        this.curso = curso;}
    public String getRa() {
        return ra; }
    public void setRa(String ra) {
        this.ra = ra; }
    public String getNome() {
        return nome; }
    public void setNome(String nome) {
        this.nome = nome; }

    @Override
    public String toString() {
        return "\n\tAluno{" +
                "id=" + id +
                ", nome='" + nome + '\'' +
                ", ra='" + ra + '\'' +
                ", curso='" + curso + '\''
+
                '}';
    }
}
```

Disciplina.java

```
package GestaoDisciplinas;
```

```
import java.util.ArrayList;

public class Disciplina {
    private int id;
    private String nome, professor;
    private ArrayList<Aluno> listaAlunos;
    private ArrayList<Avaliacao>
listaAvaliacoes;

    public Disciplina() {
        this.listaAvaliacoes = new
ArrayList<>();
        this.listaAlunos = new
ArrayList<>();
    }
    public Disciplina(int id, String nome,
String professor) {
        this.id = id;
        this.nome = nome;
        this.professor = professor;
        this.listaAvaliacoes = new
ArrayList<>();
        this.listaAlunos = new
ArrayList<>();

    public void matricularAluno(Aluno
aluno){
        this.listaAlunos.add(aluno);
        System.out.print("Aluno
matriculado.\n");
    }
    public void criarAvaliacao(int id,
String nome){
        this.listaAvaliacoes.add(new
Avaliacao(id, nome));
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getNome() {
        return nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    public String getProfessor() {
        return professor;
    }
    public void setProfessor(String
professor) {
        this.professor = professor;
    }

    public ArrayList<Avaliacao>
getListaAvaliacoes() {
        return listaAvaliacoes;
    }

    @Override
    public String toString() {
        return "Disciplina{" +
                "id=" + id +
                ", nome='" + nome + '\'' +
                ", professor='" +
professor + '\'' +
                ", alunos=" + listaAlunos
+ '\'' +
                ", avaliações= " +
listaAvaliacoes + '\'' +
                '}';
    }
}
```

Avaliacao.java

```
package GestaoDisciplinas;
import java.util.ArrayList;
public class

Avaliacao {
    private int id;
    private String nome;
    private ArrayList<Questao>
listaQuestoes;

    public Avaliacao() {
        this.listaQuestoes = new
ArrayList<>();
    }
    public Avaliacao(int id, String nome)
{
        this.id = id;
        this.nome = nome;
        this.listaQuestoes = new
ArrayList<>();
    }
    public void adicionarQuestao(int num,
String texto, float peso){
        this.listaQuestoes.add(new
Questao(num, texto, peso));
        System.out.println("Questão
adicionada.");
    }
    public int getId() {
        return id;}
    public void setId(int id) {
        this.id = id; }
    public String getNome() {
        return nome;}
    public void setNome(String nome) {
        this.nome = nome;}

    @Override
    public String toString() {
        return "\n\t\tAvaliacao{" +
                "id=" + id +
                ", nome='" + nome + '\'' +
                ", questoes= " +
listaQuestoes + '\'' +
                '}';
    }
}
```

Questao.java

```
package GestaoDisciplinas;

public class Questao {
    private int numero;
    private String enunciado;
```

```
    private float valor;

    public Questao() {
    }

    public Questao(int numero, String
enunciado, float valor) {
        this.numero = numero;
        this.enunciado = enunciado;
        this.valor = valor;
    }
    public int getNumero() {
        return numero; }
    public void setNumero(int numero) {
        this.numero = numero;}
    public String getEnunciado() {
        return enunciado; }
    public void setEnunciado(String
enunciado) {
        this.enunciado = enunciado;}
    public float getValor() {
        return valor; }
    public void setValor(float valor) {
        this.valor = valor; }
    @Override
    public String toString() {
        return "\n\t\t\tQuestao{" +
                "numero=" + numero +
                ", enunciado='" +
enunciado + '\'' +
                ", valor=" + valor +
                '}';
    }
}
```

TestaAcademico.java

```
package GestaoDisciplinas;

public class TesteAcademico {
    public static void main(String[] args)
{
        Aluno aluno1 = new Aluno(343,
"Sistemas de Informação", "Jefferson",
"1234567-3");
        Aluno aluno2 = new Aluno(200,
"Ciência da Computação", "Maria Clara",
"8584273-2");

        Disciplina disciplinaPOO = new
Disciplina(4, "Programação Orientada a
Objetos", "Daniel Facciolo");


disciplinaPOO.matricularAluno(aluno1);
disciplinaPOO.matricularAluno(aluno2);

        disciplinaPOO.criarAvaliacao(1,
"Prova Semestral");

        Avaliacao prova1 =
disciplinaPOO.getListaAvaliacoes().get(0);

        prova1.adicionarQuestao(1,
"Questão 1", 2);
        prova1.adicionarQuestao(2,
"Questão 2", 1);
        prova1.adicionarQuestao(3,
"Questão 3", 1);

System.out.println(disciplinaPOO.toString(
));

    }

}
```

Parte 1: Classe de Associação (Relacionamentos com Atributos)

Exercício 1: Sistema de Clínica Médica

Crie um sistema para gerenciar consultas.

- Classes: Medico (id, nome, especialidade) e Paciente (id, nome, cpf).
- Classe de Associação: Consulta. Ela deve conter a Data (use LocalDateTime), o ValorDaConsulta e as referências para o Médico e o Paciente.
- **Desafio:** No main, crie um médico e um paciente, e registre uma consulta para o dia 20/05/2026.

Consulta.java

```
package ClinicaMedica;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
// Classe de Associação
public class Consulta {
    private LocalDateTime data;
    private float valorConsulta;
    private Medico medico;
    private Paciente paciente;

    public Consulta() {
    }
    public Consulta(LocalDateTime data,
Paciente paciente, Medico medico, float
valorConsulta) {
        this.data = data;
        this.paciente = paciente;
        this.medico = medico;
        this.valorConsulta =
valorConsulta;
    }
```

```
    public LocalDateTime getData() {
        return data;
    }
    public void setData(LocalDateTime
data) {
        this.data = data;
    }
    public Paciente getPaciente() {
        return paciente;
    }
    public void setPaciente(Paciente
paciente) {
        this.paciente = paciente;
    }
    public Medico getMedico() {
        return medico;
    }
    public void setMedico(Medico medico) {
        this.medico = medico;
    }
    public float getValorConsulta() {
        return valorConsulta;
    }
    public void setValorConsulta(float
valorConsulta) {
        this.valorConsulta =
valorConsulta;
    }
    @Override
    public String toString() {
        DateTimeFormatter formatoBR =
DateTimeFormatter.ofPattern("dd/MM/yyyy
'às' HH'h'mm");
        return "Consulta{" +
                "data=" +
data.format(formatoBR) +
                ", valorConsulta=" +
valorConsulta +
                ", \n\tmedico=" + medico +
                ", \n\tpaciente=" +
paciente +
                '}';
    }
}
```

Medico.java

```
- package ClinicaMedica;
public class Medico {
    private int id;
    private String nome, especialidade;

    public Medico() {
    }
    public Medico(int id, String
especialidade, String nome) {
        this.id = id;
        this.especialidade =
especialidade;
        this.nome = nome;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getNome() {
        return nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    public String getEspecialidade() {
        return especialidade;
    }
    public void setEspecialidade(String
especialidade) {
        this.especialidade =
especialidade;
    }
    @Override
    public String toString() {
        return "Medico{" +
                "id=" + id +
                ", nome='" + nome + '\'' +
                ", especialidade='" +
especialidade + '\'' +
                '}';
    }
}
```

Paciente.java

```
- package ClinicaMedica;
public class Paciente {
    private int id;
    private String nome, cpf;

    public Paciente() {
    }
    public Paciente(String nome, int id,
String cpf) {
        this.nome = nome;
        this.id = id;
        this.cpf = cpf;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getCpf() {
        return cpf;
    }
    public void setCpf(String cpf) {
        this.cpf = cpf;
    }
    public String getNome() {
        return nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    @Override
    public String toString() {
        return "Paciente{" +
                "id=" + id +
                ", nome='" + nome + '\'' +
                ", cpf='" + cpf + '\'' +
                '}';
    }
}
```

TestaClinica.java

```
- package ClinicaMedica;
import java.time.LocalDateTime;
public class TestaClinica {
    public static void main(String[]
args){
        Paciente p1 = new
Paciente("Jorge", 2, "123456");
        Medico med1 = new Medico(1,
"Geral", "Pedro");
        LocalDateTime data =
LocalDateTime.of(2026,5,20,8,30);
        float valor = 10;

        Consulta consulta = new
Consulta(data, p1, med1, valor);


System.out.println(consulta.toString());
    }
}
```

Exercício 2: Sistema de Biblioteca

Modele o empréstimo de livros.

- Classes: Leitor (id, nome) e Livro (id, titulo, autor).
- Classe de Associação: Emprestimo. Deve conter a dataEmprestimo, a dataDevolucaoPrevista e o vínculo entre o Leitor e o Livro.
- **Regra:** O método toString() do empréstimo deve mostrar os dados do leitor e o título do livro emprestado.
-

Emprestimo.java

```
package SistemaBiblioteca;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
public class Emprestimo {
    private LocalDateTime dataEmprestimo,
dataDevolucao;
    private Leitor leitor;
    private Livro livro;

    public Emprestimo(){}
    public Emprestimo(LocalDateTime
dataEmprestimo, LocalDateTime
dataDevolucao, Leitor leitor, Livro
livro){
        this.dataEmprestimo =
dataEmprestimo;
        this.dataDevolucao =
dataDevolucao;
        this.leitor = leitor;
        this.livro = livro;
    }
    public LocalDateTime
getDataEmprestimo(){
        return this.dataEmprestimo;
    }
    public LocalDateTime
getDataDevolucao(){
        return this.dataDevolucao;
    }
    public Leitor getLeitor (){
        return this.leitor;
    }
    public Livro getLivro(){
        return this.livro;
    }
    public void
setDataEmprestimo(LocalDateTime
dataEmprestimo) {
        this.dataEmprestimo =
dataEmprestimo;
    }
    public void
setDataDevolucao(LocalDateTime
dataDevolucao) {
        this.dataDevolucao =
dataDevolucao;
    }
    public void setLeitor(Leitor leitor) {
        this.leitor = leitor;
    }
    public void setLivro(Livro livro) {
        this.livro = livro;
    }
    @Override
    public String toString() {
        DateTimeFormatter formatoBR =
DateTimeFormatter.ofPattern("dd/MM/yyyy");
        return "Empréstimo: " +
                "\n\t-Livro: " +
this.getLivro().getTitulo() +
                "\n\t-Autor: " +
this.getLivro().getAutor() +
                "\n\t-Leitor: " +
this.getLeitor().getNome() +
                "\n\t-Data de Empréstimo:
" +
this.getDataEmprestimo().format(formatoBR)
+
                "\n\t-Data de Devolução: "
+
this.getDataDevolucao().format(formatoBR)
    }
}
```

Leitor.java

```
- package SistemaBiblioteca;
public class Leitor {
    private int id;
    private String nome;
    public Leitor() {
    }
    public Leitor(int id, String nome) {
        this.id = id;
        this.nome = nome;
    }
    public int getId() {
```

```
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getNome() {
        return nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    @Override
    public String toString() {
        return "Leitor{" +
                "id=" + id +
                ", nome='" + nome + '\'' +
                '}';
    }
}
```

Livro.java

```
-package SistemaBiblioteca;
public class Livro {
    private int id;
    private String titulo, autor;
    public Livro() {
    }
    public Livro(int id, String autor,
String titulo) {
        this.id = id;
        this.autor = autor;
        this.titulo = titulo;
    }
    public String getTitulo() {
        return titulo;
    }
    public void setTitulo(String titulo) {
        this.titulo = titulo;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getAutor() {
        return autor;
    }
    public void setAutor(String autor) {
        this.autor = autor;
    }
    @Override
    public String toString() {
        return "Livro{" +
                "id=" + id +
                ", titulo='" + titulo +
'\'' +
                ", autor='" + autor + '\''
+
                '}';
    }
}
```

TestaBiblioteca.java

```
- package SistemaBiblioteca;
import java.time.LocalDateTime;
public class TestaBiblioteca {
    public static void main(String[]
args){
        Livro livro = new Livro(25,
"João", "A Fera");
        Leitor leitor = new Leitor(10,
"Pedro");
        LocalDateTime dataDevolucao =
LocalDateTime.of(2026,3,28,23,59);
        Emprestimo emp1 = new
Emprestimo(LocalDateTime.now(),
dataDevolucao, leitor, livro);

System.out.println(emp1.toString());
    }
}
```

Parte 2: Apenas Agregação

(Independência entre as partes)

Exercício 3: Time de Basquete (Inspirado no Franca Basquete)

- Classes: Atleta (id, nome, posicao) e Time (id, nome, tecnico).
- O Time possui um ArrayList<Atleta>.
- **Regra:** Implemente o método void contratarAtleta(Atleta a). No main, prove que se o objeto Time for anulado, o objeto Atleta continua existindo.

Atleta.java

```
package TimeBasquete;
public class Atleta {
    private int id;
    private String nome, posicao;
    public Atleta() {
    }
    public Atleta(int id, String nome,
String posicao){
        this.id = id;
        this.nome = nome;
        this.posicao = posicao;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getNome() {
        return nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    public String getPosicao() {
        return posicao;
    }
```

```
    public void setPosicao(String posicao)
{
        this.posicao = posicao;
    }
    @Override
    public String toString() {
        return "\n\tAtleta{" +
                "id=" + id +
                ", nome='" + nome + '\'' +
                ", posicao='" + posicao +
'\'' +
                '}';
    }
}
```

Time.java

```
- package TimeBasquete;
import java.util.ArrayList;
public class Time {
    private int id;
    private String nome, tecnico;
    private ArrayList<Atleta>
atletasEquipe;
    public Time(){
        this.atletasEquipe = new
ArrayList<>();
    }
    public Time(int id, String nome,
String tecnico){
        this.id = id;
        this.nome = nome;
        this.tecnico = tecnico;
        this.atletasEquipe = new
ArrayList<>();
    }
    public void contratarAtleta(Atleta
atleta){
        this.atletasEquipe.add(atleta);
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getNome() {
        return nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    public String getTecnico() {
        return tecnico;
    }
    public void setTecnico(String tecnico)
{
        this.tecnico = tecnico;
    }
    public ArrayList<Atleta>
getAtletasEquipe() {
        return atletasEquipe;
    }
    public void
setAtletasEquipe(ArrayList<Atleta>
atletasEquipe) {
        this.atletasEquipe =
atletasEquipe;
    }
    @Override
    public String toString() {
        return "Time{" +
                "id=" + id +
                ", nome='" + nome + '\'' +
                ", tecnico='" + tecnico +
'\'' +
                ", atletasEquipe=" +
atletasEquipe +
                '}';
    }
}
```

TestaBasquete.java

```
-package TimeBasquete;
public class TestaBasquete {
    public static void main(String[]
args){
        Atleta a1 = new Atleta(1, "Zé",
"Pivô");
        Time time = new Time(1,
"SesiFranca", "Pedro");

System.out.println(time.toString());
        time.contratarAtleta(a1);

System.out.println(time.toString());
        // Time anulado
        time = null;
        System.out.println(time);
        // Atleta independente
        System.out.println("Sem equipe:" +
a1.toString());
    }
}
```

Exercício 4: Projeto de Software

- Classes: Programador (id, nome, linguagemPrincipal) e Projeto (id, nomeProjeto).
- O Projeto agrega vários programadores.
- **Regra:** Crie um método para listar todos os programadores que estão trabalhando naquele projeto específico.

Programador.java

```
package ProjetoSoftware;
public class Programador {
    private int id;
    private String nome,
linguagemPrincipal;
    public Programador() {
    }
```

```
    public Programador(int id, String
nome, String linguagemPrincipal) {
        this.id = id;
        this.nome = nome;
        this.linguagemPrincipal =
linguagemPrincipal;
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getNome() {
        return nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }

    public String getLinguagemPrincipal()
{
        return linguagemPrincipal;
    }
    public void
setLinguagemPrincipal(String
linguagemPrincipal) {
        this.linguagemPrincipal =
linguagemPrincipal;
    }
    @Override
    public String toString() {
        return "Programador{" +
                "id=" + id +
                ", nome='" + nome + '\'' +
                ", linguagemPrincipal='" +
linguagemPrincipal + '\'' +
                '}';
    }
}
```

Projeto.java

```
- package ProjetoSoftware;
import java.util.ArrayList;
public class Projeto {
    private int id;
    private String nomeProjeto;
    private ArrayList<Programador>
listaProgramadores;
    public Projeto(){
        this.listaProgramadores = new
ArrayList<>();
    }
    public Projeto(int id, String
nomeProjeto){
        this.id = id;
        this.nomeProjeto = nomeProjeto;
        this.listaProgramadores = new
ArrayList<>();
    }
    public void
adicionarProgramador(Programador
programador){

this.listaProgramadores.add(programador);
    }
    public void listarProgramadores(){

if(!this.listaProgramadores.isEmpty()) {

System.out.println("Programadores:");
            for (Programador programador :
this.listaProgramadores) {
                System.out.printf("""
                        Nome:
%s\tLinguagem: %s
                        """,
programador.getNome(),
programador.getLinguagemPrincipal());
            }
        } else System.out.println("\nNão
há nenhum programador trabalhando no
projeto");
    }
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getNomeProjeto() {
        return nomeProjeto;
    }
    public void setNomeProjeto(String
nomeProjeto) {
        this.nomeProjeto = nomeProjeto;
    }
    public ArrayList<Programador>
getListaProgramadores() {
        return listaProgramadores;
    }
    public void
setListaProgramadores(ArrayList<Programado
r> listaProgramadores) {
        this.listaProgramadores =
listaProgramadores;
    }
    @Override
    public String toString() {
        return "Projeto{" +
                "id=" + id +
                ", nomeProjeto='" +
nomeProjeto + '\'' +
                ", listaProgramadores=" +
listaProgramadores +
                '}';
    }
}
```

PROJETO SOFTWARE

TestaProjeot.java

```
- package ProjetoSoftware;
public class TestaProjeto {
    public static void main(String[]
args){
        Programador p1 = new
Programador(1, "Zé", "Java");
        Programador p2 = new
Programador(2, "João", "Python");
```

```
        Programador p3 = new
Programador(3, "Pedro", "C++");
        Programador p4 = new
Programador(4, "Lucas", "Go");

        Projeto projeto1 = new Projeto(5,
"Sistema Bibliotecário");
        Projeto projeto2 = new Projeto(6,
"Clinica Médica");

        projeto1.adicionarProgramador(p1);
        projeto1.adicionarProgramador(p2);
        projeto1.adicionarProgramador(p3);
        projeto1.adicionarProgramador(p4);

        projeto1.listarProgramadores();
        projeto2.listarProgramadores();
    }
}
```

Parte 3: Apenas Composição (Dependência Existencial)

Exercício 5: Computador e Seus Componentes

- Classes: Processador (marca, modelo, frequencia) e Computador (id, marca).
- Um Computador é composto por **um** Processador.
- **Regra:** O processador deve ser instanciado **dentro** do construtor da classe Computador. Se o computador for deletado, o processador também deve ser.

Computador.java

```
package ComponentesComputador;
public class Computador {
    private int id;
    private String marca;
    private Processador processador;
    public Computador(){}
    public Computador(int id, String
marca, String marcaProcessador, String
modeloProcessador, float frequencia){
        this.id = id;
        this.marca = marca;
        this.processador = new
Processador(marcaProcessador, frequencia,
modeloProcessador);
    };
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public Processador getProcessador() {
        return processador;
    }
    public void setProcessador(Processador
processador) {
        this.processador = processador;
    }
    public String getMarca() {
        return marca;
    }
    public void setMarca(String marca) {
        this.marca = marca;
    }
    @Override
    public String toString() {
        return "Computador{" +
                "id=" + id +
                ", marca='" + marca + '\''
+
                ", processador=" +
processador +
                '}';
    }
}
```

Processador.java

```
- package ComponentesComputador;
public class Processador {
    private String marca, modelo;
    private float frequencia;
    public Processador() {
    }
    public Processador(String marca, float
frequencia, String modelo) {
        this.marca = marca;
        this.frequencia = frequencia;
        this.modelo = modelo;
    }
    public String getMarca() {
        return marca;
    }
    public void setMarca(String marca) {
        this.marca = marca;
    }
    public String getModelo() {
        return modelo;
    }
    public void setModelo(String modelo) {
        this.modelo = modelo;
    }
    public float getFrequencia() {
        return frequencia;
    }
    public void setFrequencia(float
frequencia) {
        this.frequencia = frequencia;
    }
    @Override
    public String toString() {
        return "\nProcessador{" +
```

```
                "marca='" + marca + '\'' +
                ", modelo='" + modelo +
'\'' +
                ", frequencia=" +
frequencia +
                '}';
    }
}
```

TestaComputador.java

```
- package ComponentesComputador;
public class TestaComputador {
    public static void main(String[]
args){
        Computador computador = new
Computador(5, "Alienware", "Intel",
"i5-1000", 3.5F);

System.out.println(computador.toString());
        computador = null;
        // Processador deixa de existir
junto com o Computador
    }
}
```

Exercício 6: Edifício e Apartamentos

- Classes: Apartamento (numero, andar) e Edificio (nome, endereco).
- O Edificio possui um ArrayList<Apartamento>.
- **Regra:** Implemente o método void construirApartamento(int num, int andar). Os dados do apartamento são passados como tipos primitivos, e o objeto é criado dentro do Edifício.

Apartamento.java

```
package EdificiosApartamentos;
public class Apartamento {
    private int numero, andar;
    public Apartamento() {
    }
    public Apartamento(int numero, int
andar) {
        this.numero = numero;
        this.andar = andar;
    }
    public int getNumero() {
        return numero;
    }
    public int getAndar() {
        return andar;
    }
    @Override
    public String toString() {
        return "\nApartamento{" +
                "numero=" + numero +
                ", andar=" + andar +
                '}';
    }
}
```

Edificio.java

```
- package EdificiosApartamentos;
import java.util.ArrayList;
public class Edificio {
    private int id;
    private String endereco;
    private ArrayList<Apartamento>
listaApartamentos;
    public Edificio(){
        this.listaApartamentos = new
ArrayList<>();
    }
    public Edificio(int id, String
endereco){
        this.id = id;
        this.endereco = endereco;
        this.listaApartamentos = new
ArrayList<>();
    }
    public void construirApartamento(int
numero, int andar){
        this.listaApartamentos.add(new
Apartamento(numero, andar));
    }
    public void listarApartamentos(){

if(!this.listaApartamentos.isEmpty()){
            for(Apartamento apartamento :
this.listaApartamentos){
                System.out.printf("""
                        Andar: %x\tNumero:
%x
                        """,
apartamento.getAndar(),
apartamento.getNumero());
            }
        }
    }
    @Override
    public String toString() {
        return "Edificio{" +
                "id=" + id +
                ", endereco='" + endereco
+ '\'' +
                ", listaApartamentos=" +
listaApartamentos +
                '}';
    }
}
```

TestaEdificio.java

```
- package EdificiosApartamentos;
public class TestaEdificio {
    public static void main(String[]
args){
        Edificio ed1 = new Edificio(1,
"Rua do zé");

        ed1.construirApartamento(1,1);
```

```
        ed1.construirApartamento(2,1);
        ed1.construirApartamento(5,4);
        ed1.construirApartamento(7,5);
        ed1.listarApartamentos();
        ed1 = null;
        ed1.listarApartamentos();
        // Não existe mais Edificio e
consequentemente não há mais Apartamentos
    }
}
```

Parte 4: Agregação e Composição Combinadas

Exercício 7: Sistema de Vendas de Supermercado

Este exercício combina tudo o que vimos até agora:**Agregação:** Crie a classe Produto (id, nome, preco). O Produto existe no estoque da loja independente de qualquer venda.**Agregação:** Crie a classe Cliente (id, nome).**Composição:** Crie a classe ItemVenda (id, quantidade). O item de venda só existe dentro de uma Venda.**A Classe "Todo":** Crie a classe Venda.

- Ela agrega **um** Cliente.
- Ela é composta por um ArrayList<ItemVenda>.
- Cada ItemVenda dentro da lista, por sua vez, agrega **um** Produto.

Cliente.java

```
public class Cliente {
    private int id;
    private String nome;
    public Cliente (){
    }
    public Cliente (int id, String nome){
        this.id = id;
        this.nome = nome;
    }
    public int getId() {
        return id;
    }
    public String getNome() {
        return nome;
    }
}
```

ItemVenda.java

```
- public class ItemVenda {
    private int id, quantidade;
    private Produto produto;
    public ItemVenda(){
    }

    public ItemVenda(int id, int
quantidade, Produto produto){
        this.id = id;
        this.quantidade = quantidade;
        this.produto = produto;
    }
    public int getId() {
        return id;
    }
    public int getQuantidade() {
        return quantidade;
    }
    public Produto getProduto(){
        return this.produto;
    }
}
```

Produto.java

```
- public class Produto {
    private int id;
    private String nome;
    private float preco;
    public Produto(){}
    public Produto(int id, String nome,
float preco){
        this.id = id;
        this.nome = nome;
        this.preco = preco;
    };
    public int getId() {
        return id;
    }
    public String getNome() {
        return nome;
    }
    public float getPreco() {
        return preco;
    }
    public String toString(){
        return "\nProduto: " +
                "\n\t-Nome: " +
this.getNome() +
                "\n\t-ID: " + this.getId()
+
                "\n\t-Preço: " +
this.getPreco();
    }
}
```

Venda .java

```
-import java.util.ArrayList;
public class Venda {
    private Cliente cliente;
    private ArrayList<ItemVenda>
itensVenda;
    public Venda() {
```

```
        this.itensVenda = new
ArrayList<>();
    }
    public Venda(Cliente cliente){
        this.cliente = cliente;
        this.itensVenda = new
ArrayList<>();
    }
    public void adicionarItem(int id, int
quantidade, Produto produto){
        this.itensVenda.add(new
ItemVenda(id, quantidade, produto));
    }
    private String listarItens(){
        String itens = "[";
        for(ItemVenda itemVenda :
this.itensVenda){
            if
(!itens.equals("[")){itens+= ", ";};
            itens +=
itemVenda.getProduto().getNome() + "(" +
itemVenda.getId() + ")";
        }

        return itens + "]";
    }
    public String toString(){
        return "* Venda *" +
                "\nCliente: " +
this.cliente.getNome() +
                "\nProdutos: " +
this.listarItens();
    }
}
```

TestaSupermercado.java

```
- public class TestaSupermercado {
    public static void main(String[]
args){
        Cliente cliente1 = new Cliente(1,
"João");
        Venda venda1 = new
Venda(cliente1);
        Produto p1 = new Produto(1,
"Feijão", 5);
        Produto p2 = new Produto(2,
"Arroz", 6);
        Produto p3 = new Produto(3,
"Macarrão", 5);
        Produto p4 = new Produto(4,
"Bolacha", 1);
        venda1.adicionarItem(1, 1, p1);
        venda1.adicionarItem(2, 2, p2);
        venda1.adicionarItem(3, 5, p3);
        venda1.adicionarItem(4, 10, p4);

System.out.println(venda1.toString());
        venda1 = null;
        System.out.println(p1.toString());
        System.out.println(p2.toString());
        // Os itens da venda são perdidos

System.out.println(venda1.toString());
    }
}
```

Exercício 8: Sistema de Cine-Teatro

Objetivo: Criar um sistema que gerencie uma Sessao, relacionando-a com um Filme (Agregação) e com os Ingressos vendidos (Composição).1. Classe Filme (Objeto Independente)
Um filme existe no catálogo do cinema, independentemente de estar em cartaz ou não.

- **Atributos:** id, titulo, genero, duracao (int - em minutos).
- **Requisito:** Construtor completo, Getters/Setters e toString().

2. Classe Ingresso (Objeto Parte)
Um ingresso só faz sentido dentro de uma sessão específica. Se a sessão for cancelada, o ingresso perde o valor e deixa de existir no sistema.

- **Atributos:** id, assento (String), tipo (String: "Inteira" ou "Meia"), preco (float).
- **Requisito:** Construtor completo e toString().

3. Classe Sessao (O "Todo" Principal)
A Sessao é o coração do sistema, onde as relações acontecem.

- **Atributos:** id, horario (use LocalDateTime), sala (int).
- **Relacionamentos:**
- **Agregação:** Um atributo do tipo Filme. (Use o método vincularFilme(Filme f) que recebe o objeto pronto).
- **Composição:** Um ArrayList<Ingresso>. (Use o método venderIngresso(int id, String assento, String tipo, float preco) que cria o objeto Ingresso internamente).

4. Classe de Teste TestaCinema

No método main, realize o fluxo:Instancie 2 objetos Filme (ex: "Batman", "Duna").Instancie uma Sessao para a Sala 01, informando uma data e hora futura (ex: 20/07/2026 às 20h00).**Agregação:** Vincule o filme "Batman" a esta sessão.**Composição:** Simule a venda de 3 ingressos (A1, A2 e A3) usando o método de composição da classe Sessao.Imprima os dados da Sessão.

Filme.java

```
package CineTeatro;
public class Filme {
    private int id, duracao;
    private String titulo, genero;
    public Filme(int id, String titulo,
String genero, int duracao){
        this.id = id;
        this.titulo = titulo;
        this.genero = genero;
        this.duracao = duracao;
    }
    public int getId(){
        return this.id;
    }
    public String getTitulo(){
        return this.titulo;
    }
    public String getGenero(){
        return this.genero;
    }
    public int getDuracao(){
        return this.duracao;
    }
    public void setId(int id){
        this.id = id;
    }
    public void setTitulo(String titulo){
        this.titulo = titulo;
    }
    public void setGenero(String genero){
        this.genero = genero;
    }
    public void setDuracao(int duracao){
        this.duracao = duracao;
    }
    @Override
    public String toString() {
        return "Filme{" +
                "id=" + id +
                ", duracao=" + duracao +
                ", titulo='" + titulo +
'\'' +
                ", genero='" + genero +
'\'' +
                '}';
    }
}
```

Ingresso.java

```
- package CineTeatro;

public class Ingresso {
    private int id;
    private String assento, tipo;
    private float preco;

    public Ingresso(int id, String
assento, String tipo, float preco){
        this.id = id;
        this.assento = assento;
        this.tipo = tipo;
        this.preco = preco;
    }
    @Override
    public String toString() {
        return "ID: "+ this.id+
                "\tAssento: " +
this.assento+
                "\tTipo: " + this.tipo;
    }
}
```

Sessao.java

```
- package CineTeatro;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Date;

public class Sessao {
    private int id, sala;
    private LocalDateTime horario;
    private Filme filme;
    private ArrayList<Ingresso>
listaIngressos;

    public Sessao(int id, int sala,
LocalDateTime horario){
        this.id = id;
        this.sala = sala;
        this.horario = horario;
        this.listaIngressos = new
ArrayList<>();
    }
    public void vincularFilme(Filme
filme){
        this.filme = filme;
    }
    public void venderIngresso(int id,
String assento, String tipo, float preco){
        this.listaIngressos.add(new
Ingresso(id, assento, tipo, preco));
    }
    public void listarIngressos(){
        System.out.println("\nIngressos
vendidos para " + this.filme.getTitulo());
        for(Ingresso ingresso :
this.listaIngressos){
            System.out.println("\t"+
ingresso.toString());
        }
```

```
    }
    public String toString(){
        DateTimeFormatter formatoBR =
DateTimeFormatter.ofPattern("dd/MM/yyyy -
HH'h'mm");
        return "\n* Sessão *" +
                "\n\tFilme: " +
this.filme.getTitulo() +
                "\n\tGênero: " +
this.filme.getGenero() +
                "\n\tDuração: " +
this.filme.getDuracao() +
                "\n\tHorário: " +
this.horario.format(formatoBR) +
                "\n\tIngressos vendidos: "
+ this.listaIngressos.size();
    }
}
```

TestaCinema.java

```
- package CineTeatro;

import java.lang.invoke.SwitchPoint;
import java.time.LocalDateTime;

public class TestaCinema {
    public static void main(String[]
args){
        Filme f1 = new Filme(1, "Batman",
"Ação/Aventura", 120);
        Filme f2 = new Filme(2, "Duna",
"Ficção Científica", 170);

        Sessao sessao = new Sessao(1, 1,
LocalDateTime.of(2026, 7, 20, 20,0));

        sessao.vincularFilme(f1);


System.out.println(sessao.toString());

        sessao.venderIngresso(1, "H3",
"Meia", 15);
        sessao.venderIngresso(2, "H4",
"Meia", 15);
        sessao.venderIngresso(3, "K7",
"Inteira", 29);


System.out.println(sessao.toString());

        sessao.listarIngressos();
    }
}
```

EXE HERANÇA, POLIMORFISMO E CLASSE ABSTRATA

Exercício 1: Especialização de Comportamento (Sistema de Veículos)

Objetivo: Praticar a criação de uma nova hierarquia e a anulação de métodos com regras de negócio distintas.

Enunciado:

Modele um sistema de transporte seguindo as instruções abaixo:**Classe Pai Veiculo**:

- Atributos: marca (String), modelo (String), velocidade (float).
- Método mover(): Deve imprimir "Veículo se movendo...".
- Método abastecer(): Deve imprimir "Veículo reabastecendo...".

Classe Filha Aviao:

- Atributo próprio: altitudeMax (float).
- Anule o método mover(): Deve imprimir "Avião voando a [velocidade] km/h e [altitude] metros".
- Anule o método abastecer(): Deve imprimir "Abastecendo com querosene de aviação".

Classe Filha CarroEletrico:

- Atributo próprio: autonomiaBateria (int).
- Anule o método mover(): Deve imprimir "Carro elétrico movendo-se silenciosamente".
- Anule o método abastecer(): Deve imprimir "Recarregando bateria em posto de carga rápida".

Classe de Teste:

- Crie instâncias das classes.
- Crie uma variável do tipo Veiculo e faça-a apontar para um Aviao. Chame o método mover().
- Mude a mesma variável para apontar para um CarroEletrico. Chame o método mover() novamente.

aviao.java

```
package exe1;

public class Aviao extends Veiculo {
    private float altitudeMax;

    public Aviao() {
        super();
    }

    public Aviao(String marca, String
modelo, float velocidade, float
altitudeMax) {
        super(marca, modelo, velocidade);
        this.altitudeMax = altitudeMax;
    }

    @Override
    public void mover() {
        System.out.println("Avião voando a
" + this.velocidade + " km/h e " +
this.altitudeMax + " metros de
altitude.");
    }

    @Override
    public void abastecer() {
        System.out.println("Abastecendo
com querosene de aviação.");
    }

    @Override
    public String toString() {
        return "Aviao{" +
                super.toString() +
                "altitudeMax=" +
altitudeMax +
                '}';
    }
}
```

CarroEletrico.java

```
package exe1;

public class CarroEletrico extends Veiculo
{
    private int autonomiaBateria;

    public CarroEletrico(){
        super();
    }
    public CarroEletrico(String marca,
String modelo, float velocidade, int
autonomiaBateria) {
        super(marca, modelo, velocidade);
        this.autonomiaBateria =
autonomiaBateria;
    }

    @Override
    public void mover() {
        System.out.println("Carro elétrico
movendo-se silenciosamente. Autonomia: " +
this.autonomiaBateria + "km.");
    }

    @Override
    public void abastecer() {
        System.out.println("Recarregando
bateria em posto de carga rápida.");
    }

    public int getAutonomiaBateria() {
        return autonomiaBateria;
    }

    public void setAutonomiaBateria(int
autonomiaBateria) {
        this.autonomiaBateria =
autonomiaBateria;
    }

    @Override
    public String toString() {
        return "CarroEletrico{" +
                super.toString() +
                "autonomiaBateria=" +
autonomiaBateria +
                '}';
    }
}
```

Veiculo.java

```
package exe1;

public abstract class Veiculo {
    protected String marca, modelo;
    protected float velocidade;

    public Veiculo() {
    }

    public Veiculo(String marca, String
modelo, float velocidade) {
        this.marca = marca;
        this.modelo = modelo;
        this.velocidade = velocidade;
    }

    public abstract void mover();

    public abstract void abastecer();

    // Getters e Setters podem ser
adicionados aqui se desejar

    public String getMarca() {
        return marca;
    }

    public void setMarca(String marca) {
        this.marca = marca;
    }

    public String getModelo() {
        return modelo;
    }

    public void setModelo(String modelo) {
        this.modelo = modelo;
    }
```

```
    public float getVelocidade() {
        return velocidade;
    }

    public void setVelocidade(float
velocidade) {
        this.velocidade = velocidade;
    }

    @Override
    public String toString() {
        return "Veiculo{" +
                "marca='" + marca + '\'' +
                ", modelo='" + modelo +
'\'' +
                ", velocidade=" +
velocidade +
                '}';
    }
}
```

TestaTransporte.java

```
package exe1;
public class TestaTransporte {
    public static void exibe(Veiculo
referencia){
        referencia.mover();      //
Polimorfismo:
        referencia.abastecer();  //
Polimorfismo:
    }
    public static void main(String[] args)
{
         // Fazendo a variável apontar
para um objeto FILHO (Aviao)
        System.out.println("--- Testando
Avião ---");
        Aviao av1 = new Aviao("Boeing",
"747", 900.0f, 11000.0f);
        exibe(av1);
        System.out.println("\n--- Testando
Carro Elétrico ---");
        // Mudando a mesma variável para
apontar para outro objeto FILHO
(CarroEletrico)
        CarroEletrico ce1 = new
CarroEletrico("Tesla", "Model S", 120.0f,
600);
        exibe(ce1);

    }
}
```

Exercício 2: Polimorfismo com Vetores (Folha de Pagamento)

Objetivo: Demonstrar que um vetor do tipo da superclasse pode armazenar objetos de diferentes subclasses e executar o método correto em tempo de execução.

Enunciado:

Utilizando a estrutura de classes Desenvolvedor, Junior, Pleno e Senior que fizemos em sala, crie uma classe chamada TestaVetorPolimorfico. No método main, realize os seguintes passos:Crie um **vetor (array)** de tamanho 4 do tipo Desenvolvedor.Preencha o vetor com:

- Um objeto Junior.
- Um objeto Pleno.
- Um objeto Senior.
- Um objeto Desenvolvedor (genérico).

Utilizando um laço de repetição (for ou foreach), percorra o vetor e, para cada objeto:

- Chame o método codar().
- Imprima o resultado do método calcularBonus().
- Imprima o toString() do objeto.

desenvolverdor.java

```
package FolhaPagamento;

public class Desenvolvedor{
    protected String nome, linguagem;
    protected float salarioBase;

    public Desenvolvedor() {
    }

    public Desenvolvedor(String nome,
String linguagem, float salario) {
        this.nome = nome;
        this.salarioBase = salario;
        this.linguagem = linguagem;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public float getSalario() {
        return salarioBase;
    }

    public void setSalario(float salario)
{
        this.salarioBase = salario;
    }
```

```
    public String getLinguagem() {
        return linguagem;
    }

    public void setLinguagem(String
linguagem) {
        this.linguagem = linguagem;
    }

    @Override
    public String toString() {
        return "Desenvolvedor{" +
                "nome='" + nome + '\'' +
                ", linguagem='" +
linguagem + '\'' +
                ", salario=" + salarioBase
+
                '}';
    }

    public void codar(){
        System.out.println("Desenvolvedor
planeja e escreve código-fonte");
    }
    public float calcularBonus(){
        return this.salarioBase * 0.05f;
    }
}
```

Junior.java

```
package FolhaPagamento;

public class Junior extends Desenvolvedor
{
    private String mentor;

    public Junior() {
        super(); // chama o construtor da
superClasse
    }

    public Junior(String nome, String
linguagem, float salarioBase, String
mentor){
        super(nome, linguagem,
salarioBase);
        this.mentor = mentor;
    }

    public String getMentor() {
        return mentor;
    }

    public void setMentor(String mentor) {
        this.mentor = mentor;
    }

    @Override // anulação de método
    public void codar(){
        System.out.println("Junior
desenvolvendo código com mentoria de " +
this.mentor);
    }

    @Override
    public String toString() {
        return "Junior{" +
                super.toString() +
                "mentor='" + mentor + '\''
+
                '}';
    }
}
```

Pleno.java

```
package FolhaPagamento;

public class Pleno extends Desenvolvedor {
    private int projetosEntregues;

    public Pleno(){
        super();
    }

    public int getProjetosEntregues() {
        return projetosEntregues;
    }

    public void setProjetosEntregues(int
projetosEntregues) {
        this.projetosEntregues =
projetosEntregues;
    }

    public Pleno(String nome, String
linguagem, float salarioBase, int
projetosEntregues){
        super(nome, linguagem,
salarioBase);
        this.projetosEntregues =
projetosEntregues;
    }

    @Override
    public void codar(){
        System.out.println("Pleno
desenvolvendo código e fazendo Code
Review");
    }

    @Override
    public float calcularBonus(){
        return super.calcularBonus() +
this.salarioBase * 0.15f;
    }

    @Override
    public String toString() {
        return "Pleno{" +
                super.toString() +
                "projetosEntregues=" +
projetosEntregues +
                '}';
    }
}
```

Senior.java

```
package FolhaPagamento;

public class Senior extends Desenvolvedor
{
    private float verbaLider;

    public Senior(){
        super();
    }

    public float getVerbaLider() {
        return verbaLider;
    }

    public void setVerbaLider(float
verbaLider) {
        this.verbaLider = verbaLider;
    }

    public Senior(String nome, String
linguagem, float salarioBase, float
verbaLider){
        super(nome, linguagem,
salarioBase);
        this.verbaLider = verbaLider;
    }

    @Override
    public void codar(){
        System.out.println("Sênior
definindo a arquitetura do software e
liderando pessoas");
    }

    @Override
    public float calcularBonus(){
        return super.calcularBonus() +
this.salarioBase * 0.30f;
    }

    @Override
    public String toString() {
        return "Senior{" +
                super.toString() +
                "verbaLider=" + verbaLider
+
                '}';
    }
}
```

TestaDesenvolvedor.java

```
import java.util.ArrayList;

public class TestaDesenvolvedor {

    public static void
mostra(Desenvolvedor camaleao){
        camaleao.codar(); // executa o
codar() de quem o camameao representa na
chamada do método
        // executa o toString() e
calcularBonus() de quem o camameao
representa na chamada do método

System.out.println(camaleao.toString() + "
Bônus " + camaleao.calcularBonus());
    }

    public static void main(String[] args)
{
        // cria um objeto da class
Desenvolvedor (classe pai)

        Junior jr1 = new Junior("Fulano",
"Java", 5000, "Beltrano");
        mostra(jr1);
        Pleno pl1 = new Pleno("Beltrano",
"Java", 10000, 7);
        mostra(pl1);
        Senior se1 = new Senior("Ciclano",
"Java", 12000, 3000);
        mostra(se1);

        // cria um vetor de Desenvolvedor
        ArrayList<Desenvolvedor>
desenvolvedores = new
ArrayList<Desenvolvedor>();
        // adicionar os desenvolvedores no
array
        desenvolvedores.add(jr1);
        desenvolvedores.add(pl1);
        desenvolvedores.add(se1);

        // exibe codar, calcularBonus e
toString de cada desenvolvedor
        for (Desenvolvedor dev :
desenvolvedores) {
            mostra(dev);
        }
    }
}
```

TestaVetorPolimorfico.java

```
package FolhaPagamento;

import java.sql.SQLOutput;
import java.util.ArrayList;

public class TestaVetorPolimorfico {
    public static void main(String[]
args){
        ArrayList<Desenvolvedor> vetor =
new ArrayList<>();

        vetor.add(new Junior("João",
"Java", 15, "Zeca"));
        vetor.add(new Pleno("Pedro", "C#",
30, 1));
        vetor.add(new Senior("Roberto",
"Python", 50, 100));
```

```
        vetor.add(new
Desenvolvedor("Junior", "PHP", 20));

        vetor.forEach(dev -> {
            dev.codar();
            System.out.println("Bônus "+
dev.calcularBonus());

System.out.println(dev.toString() + "\n");
        });

    }
}
```

EXE CLASSE INTERFACE E DIAGRAMA

Exercício: Sistema de Casa Inteligente (Smart Home)

Objetivo: Trabalhar com múltiplas interfaces e a obrigatoriedade de implementar diferentes contratos de comportamento em uma mesma classe.
1. As Interfaces (Os Contratos)

- **Interface Autenticavel:**
- Método: void validarAcesso(String senha);

Interface DispositivoLigavel:
- Método: void ligar();
- Método: void desligar();

Interface SensorTemperatura:
- Método: double lerTemperatura();

2. As Classes (As Implementações)
- **Classe FechaduraEletronica:**
- Deve implementar: Autenticavel e DispositivoLigavel.
- *Lógica:* Ao validar a acesso com a senha correta, ela permite "ligar" (destrancar).

Classe TermostatoSmart:
- Deve implementar: DispositivoLigavel e SensorTemperatura.
- *Lógica:* Ele pode ser ligado/desligado e retorna a temperatura atual do ambiente.

Autenticavel.java

```
public interface Autenticavel{
    void validarAcesso(String senha);

}
```

DispositivoLigavel.java

```
public interface DispositivoLigavel {
    void ligar();

    void desligar();
}
```

FechaduraEletronica.java

```
public class FechaduraEletronica
implements Autenticavel,
DispositivoLigavel{
    private String senha;
    private boolean autenticacao;

    public FechaduraEletronica(String
senha){
        this.senha = senha;
        this.autenticacao = false; //
"desligado"
    }

    public void validarAcesso(String
senha){
        if(senha.equals(this.senha)){
            this.autenticacao = true;
            System.out.println("Acesso
validado");
        } else System.out.println("Senha
incorreta");
    }

    public void ligar() {
        if (this.autenticacao) {

System.out.println("Destrancado");
        } else {
            System.out.println("Não
autenticado");
        }
    }

    public void desligar(){
        if (this.autenticacao) {

System.out.println("Trancado");
        } else {
            System.out.println("Não
autenticado");
        }
```

```
    }
}
```

SensorTemperatura.java

```
public interface SensorTemperatura {
    double lerTemperatura();
}
```

TermostatoSmart.java

```
public class TermostatoSmart implements
DispositivoLigavel, SensorTemperatura{
    private double temperatura;
    private boolean status;

    public TermostatoSmart(double
temperatura){
        this.temperatura = temperatura;
        this.status = true; // true =
ligado
    }

    public double lerTemperatura(){
        if(this.status) {
            return this.temperatura;
        } else {
            System.out.println("Termostato
desligado");
            return 0.0;
        }
    }

    public void ligar(){
        this.status = true;
        System.out.println("Termostato
ligado");
    }

    public void desligar(){
        this.status = false;
        System.out.println("Termostato
desligado");
    }

}
```

Teste.java

```
public class Teste {
    public static void main(String[]
args){
        FechaduraEletronica fechadura =
new FechaduraEletronica("1234");
        TermostatoSmart termostato = new
TermostatoSmart(20);

        fechadura.validarAcesso("1234");
        fechadura.ligar();
        fechadura.desligar();
        System.out.printf("%nTemperatura
atual: %.1f°C%n",
termostato.lerTemperatura());;
        termostato.desligar();

    }

}
```